Ultimate Basic v1.6.6 — Language Manual

Complete language and CLI reference for Ultimate Basic, a BASIC-like language that compiles directly to 6502 machine code for the Commodore 64. Output: `.prg` files (VICE or real hardware), `.crt` cartridge images, and `.d64` disk images.

For a short project overview and build instructions see README.md.

© 2026 Zsolt Tarczali

> Building the `ub` compiler and the command-line options are covered in README.md. This manual documents the Ultimate Basic language itself.

Language reference

Variables and constants

All variables must be declared with `var` before use. Using an undeclared variable in an expression, assignment, `for` / `loop` counter, `inc` / `dec`, `input`, or `read` statement produces a compile-time error.

```
var x = 10               # 8-bit integer (default)
var ptr: word = $0400    # 16-bit — two zero-page bytes, usable as 16-bit address
var f: float = 3.5       # Q8.8 fixed-point — hi byte = integer, lo byte = fraction
var msg = "HELLO"        # string variable (pointer to inline PETSCII data)
var s: string = "TEXT"   # string with explicit type
var scores = array(10)   # byte array, 10 elements stored at $C000+
var moments  = array_word(8) # word array, 8 word elements stored at $C000+
const BORDER_ADDR = $D020 # compile-time constant (substituted inline, no ZP slot)
```

Keywords and identifiers are **case-insensitive**: `PRINT`, `Print`, and `print` are all valid.

Type	Width	Notes
`int`	8-bit	default for numeric literals
`word`	16-bit	two ZP bytes; can be used as address in `poke` / `peek`
`float`	16-bit Q8.8	hi byte = integer part (0–255), lo byte = fractional part
`single` / `double`	5 bytes, RAM	floating point (C64 BASIC format, ~9 digits) — `dim x as double`; see *Floating point*
`string`	pointer	ZP pair → null-terminated PETSCII in code segment
`array(N)`	N bytes	byte elements; lives at `$C000+`, not in ZP
`array_word(N)`	N×2 bytes	word (16-bit) elements; lives at `$C000+`, not in ZP

Type	Width	Notes
`array(R, C, …)`	∏dims bytes	multi-dimensional (row-major); index `arr[r, c]`
`array_word(R, C, …)`	∏dims×2 bytes	multi-dimensional word array (row-major)

BASIC-style declarations: `dim … as …`

`dim name as type` is an alternative spelling of `var name: type` — same storage, same code:

```
DIM kor AS INTEGER          # signed 16-bit, starts at 0
DIM nev AS STRING           # string, starts as ""
DIM fizetes AS DOUBLE       # floating point, starts at 0
DIM x, y AS BYTE            # several names, all get the type
DIM pont AS INTEGER = 1000, cim AS STRING = "C64"   # initializers, mixed types
```

`as …`	Ultimate Basic type
`byte`, `int`	`int` (8-bit, 0–255)
`integer`	signed 16-bit (−32768 … 32767) — QBasic INTEGER
`word`	`word` (unsigned 16-bit, 0–65535)
`single`, `double`	floating point, 5 bytes in RAM ($\pm 2.9 \cdot 10^{-39}$ … $1.7 \cdot 10^{38}$, ~9 digits) — see *Floating point*
`float`	`float` (Q8.8 fixed point — 0–255.99, fast)
`string`	`string`

- Without `= value`, numbers start at 0 and strings at `""`.
- `dim a, b as integer` gives **both** names the type (VB.NET style, not QBasic). Only a single name may have an initializer.
- `as` and the BASIC type names are context words, not reserved — they remain usable as names.
- Not supported: `as long`, struct types (use `var name: TName = array(N)`) — compile-time errors.
- With `--explicit`, a `dim` without `as` is an error.
- String assignment has value semantics: `s = "TEXT"` points at the literal, `t = s` and computed values are copied into `t`'s own buffer (see *Strings at runtime*). Before 1.6.3, `s = "TEXT"` after the declaration stored a garbage byte into the pointer.

Arrays with `dim`: `dim name(u1, u2, …) as type`

```
DIM tabla(2, 3) AS INTEGER     ' 3 × 4 elements: tabla(0..2, 0..3)
DIM kocka(1, 2, 3) AS BYTE     ' any number of dimensions
DIM ar(9) AS DOUBLE            ' 10 floating-point values (5 bytes each)
DIM nevek(4) AS STRING         ' 5 strings, all start as ""
```

```
DIM v(9)                        ' no AS → byte array

tabla(2, 3) = 999               ' QBasic-style index with ( )
tabla[2, 3] = 999               ' … or the usual [ ]
nevek(0) = "ZSOLT"
PRINT tabla(2, 3); " "; nevek(0); " "; ar(1)
```

- Every bound is the **highest index** (BASIC rule): `dim t(2, 3)` has 3 × 4 elements. (`var t = array(3, 4)` takes element counts — same array.)
- Element types: `byte` / `int` → 1 byte, `integer` (signed) / `word` → 2 bytes, `float` → 2-byte Q8.8, `single` / `double` → 5-byte floating point, `string` → 2-byte pointer. Bounds must be compile-time constants. Arrays live in `$C000-$CFFF` (4 KB); when they need more, the whole array area moves to `$A000-$CFFF` (12 KB, under the BASIC ROM — see *Array memory*). More than 12288 bytes is a compile-time error.
- String-array elements start as `""`; `nevek(i) = "TEXT"` / `= s` / `= a + b`, `n = nevek(i)`, printing, `+`, passing to a `string` parameter and `input nevek(i)` all work (non-literal values are copied into a 31-character slot per element).
- `name(i)` indexing also works for arrays declared with `var a = array(…)`.
- One array per `dim` group: `dim a(3) as byte, n as integer` is fine, `dim n, a(3) as integer` is an error. No initializer — use `data name: …`.

Reserved words

The following identifiers are **keywords** — they cannot be used as variable, constant, subroutine, function, parameter, or label names. All matching is case-insensitive (`END`, `end`, `End` all collide). Using a reserved word as a name usually produces a confusing error (a `var` line silently fails to declare, or an expression like `for i = 1 to times` folds to `Number(0)`) — so pick a different name.

Declaration & control flow
`var`, `dim`, `const`, `sub`, `fn`, `function`, `type`, `do`, `elseif`, `wend`, `endtype`, `return`, `call`, `label`, `goto`, `gosub`,
`if`, `then`, `else`, `end`, `select`, `case`,
`for`, `next`, `loop`, `times`, `to`, `step`, `while`, `repeat`, `until`,
`break`, `continue`, `inc`, `dec`, `bye`, `exit`, `rem`

Music
`tune` (block keyword; the lines inside - `speed`, `inst`, `order`, `pat` - are not reserved)

Types & type-related
`int`, `word`, `float`, `string`, `array`, `array_word`

Print & I/O
`print`, `spc`, `tab`, `at`, `input`, `chr$`, `str$`, `hex`, `bin`,
`open`, `close`, `load`, `save`, `chain`, `data`, `read`, `include`, `incbin`
(also `dec` — listed above as the decrement statement; the same token is used for the `dec(n, width)` print format)

Math & string built-ins
`abs`, `min`, `max`, `clamp`, `sgn`, `mod`, `rnd`, `sin`, `cos`,

`and`, `or`, `xor`, `not`, `bnot`, `shl`, `shr`,
`len`, `asc`, `val`, `str_to_int`, `numstr`

Memory & timing
`poke`, `peek`, `poke16`, `peek16`, `fill`, `memcopy`, `drawmem`,
`wait`, `raster`, `delay`, `sys`, `asm`

Screen & text
`cls`, `fast`, `color`, `text`, `border`, `bg`, `screen`, `cursor`,
`lowercase`, `uppercase`, `display`, `on`, `off`, `scroll`,
`speed`, `badlines`, `turbo`

Bitmap & block graphics
`graphics`, `gcls`, `flip`, `plot`, `plot4`, `mplot`,
`mline`, `mrect`, `mcircle`,
`line`, `circle`, `circle4`, `rect`, `paint`, `erase`,
`pen`, `multi`, `block`

Sprites
`sprite`, `sprdef`, `sprite_frame`, `sprite_x`, `sprite_y`, `sprhit`, `sprbghit`,
`box_hit`, `chardef`, `charset`, `org`, `expand`, `priority`

Sound & music
`sid`, `sound`, `sfx`, `volume`, `music`, `play`, `pause`, `resume`, `stop`

Input devices
`getch`, `inkey`, `waitkey`, `joy`, `mouse_x`, `mouse_x_hi`, `mouse_y`, `mouse_btn`

Interrupts & vectors
`irq`, `irq_exit`, `nmi`, `nmi_exit`, `cia_timer`, `onerr`

Character maps & images
`map`, `map_tile`, `map_color`, `koala`, `show`, `hide`

REU (RAM expansion)
`reu`, `reudet`, `stash`, `fetch`

Style pitfalls — names that *look* free but are reserved

These common English words look like innocent identifiers but are already grabbed by the lexer. Rename to avoid silent breakage:

Reserved	Suggested rename
`end`	`stop_at`, `finish`, `last`
`stop`	`stop_at`, `halt`
`times`	`count`, `n`, `iters`
`screen`	`scraddr`, `screen_addr`
`border`, `bg`, `text`	`border_addr`, `bg_col`, `text_col`
`clamp`	`cap`, `bound`
`line`, `circle`, `rect`	`ln`, `circ`, `box_r`

Reserved	Suggested rename
`data`, `read`	`bytes`, `next_byte`
`load`, `save`, `open`, `close`	`load_file`, …
`map`, `show`, `hide`	`tilemap`, `reveal`, `conceal`
`play`, `pause`, `resume`, `stop`, `volume`	`play_song`, `vol`
`int`, `float`, `word`, `string`	`n_int`, `speed_f`, `addr`, `msg`

Symbolic tokens (`+ - * / = == != < > <= >= : , ; ( ) [ ] # $ % @`) are naturally not usable in identifiers.

Comments

```
# hash comment
' QBasic-style comment (1.6.3)
rem this is also a comment
var x = 5  # inline comment
var x = 5 : var y = 6  # colon separates statements on one line
```

Operators

```
x = x + 1
y = a * b - c / 2
z = x and 15            # bitwise AND
w = a or b              # bitwise OR
v = a xor b             # bitwise XOR
m = x shl 3             # shift left
n = x shr 2             # shift right (arithmetic for an INTEGER: -8 shr 1 = -4)
r = x mod 40            # remainder; x mod 0 = x, x / 0 = 255
b = bnot x              # bitwise NOT: x XOR 255 (complement all 8 bits)
```

Comparisons: `==` `!=` `<` `>` `<=` `>=` (return 1/0)

Arithmetic code (1.6.0): `*` by a constant compiles to shifts / shift-add (`x * 8` → 3× `ASL`), otherwise to a shift-add loop of at most 8 rounds. `/` and `mod` by a power of two are `LSR` / `AND #(n-1)`, otherwise an 8-round shift-subtract loop whose run time no longer depends on the quotient. Division by zero is defined: `x / 0 = 255`, `x mod 0 = x` (before 1.6.0 it hung).

`mod` with a `word` or `integer` operand (or a negative constant) is a 16-bit remainder with the sign of the dividend, as in QBasic: `-7 mod 3` = -1, `7 mod -3` = 1, `(w + 5) mod 7` keeps 16 bits. With a SINGLE / DOUBLE operand it works in floating point (see *Floating point*).

`/` with a `word` / `integer` or negative divisor is a 16-bit division, truncated toward zero (`-700 / 300` = -2, `-700 / -7` = 100).
`shr` of an `integer` is an arithmetic shift (the sign is kept: `-8 shr 1` = -4); `word` and byte values shift logically.
`x = x + 1` / `x = x - 1` (and ±2) compile to `INC` / `DEC`.

Increment / Decrement

```
inc x                   # x = x + 1  (INC zp — single instruction)
dec x                   # x = x - 1  (DEC zp — single instruction)
```

For `word` variables carry is handled: `inc` uses `INC lo; BNE skip; INC hi`; `dec` uses `LDA lo; BNE skip; DEC hi; DEC lo`.

Compound Assignments

```
x += 5                  # x = x + 5
x -= 3                  # x = x - 3
x *= 2                  # x = x * 2
x /= 4                  # x = x / 4
x and= 15               # x = x and 15   (bitwise AND)
x or= 64                # x = x or 64    (bitwise OR)
x xor= 255              # x = x xor 255  (bitwise XOR)
x shl= 2                # x = x shl 2
x shr= 1                # x = x shr 1
```

Print

```
print "HELLO"
print x
print x, y, "text"
print                    # blank line

print spc(5)             # print 5 space characters
print tab(20), "VALUE"  # move cursor to column 20, then print
print "A", spc(3), "B" # mix freely

print "A=" + a           # string + numeric var
print s1 + s2            # two string vars → sequential print
```

```
print "Hello " + "World" # two literals → compile-time fold
print chr$(13)           # print by PETSCII code
```

By default numbers are printed without extra spaces and `,` just separates the items (`print 1, 2` gives `12`). For C64 BASIC spacing build with `--basic-print`, or put a line `option basic print` in the program: a number then prints with a leading space (or its `-`) and a trailing space (`print 5` → `5`), a `,` moves to the next 10-column print zone, and a trailing `,` keeps the cursor on the line like a trailing `;`. `STR$` then gives the C64 text too (`STR$(7)` = `" 7"`, `STR$(-42)` = `"-42"`). Strings, `spc`, `tab`, `hex`, `bin`, `dec` and `chr$` print as before.

`PRINT USING`

```
DIM ar AS DOUBLE = 1234.5
DIM s AS STRING = "ALMA"
PRINT USING "###.##"; 3.14159            '   3.14
PRINT USING "Ar: #,###.## Ft"; ar        ' Ar: 1,234.50 Ft
PRINT USING "+##"; 7                     ' +7
PRINT USING "##.##-"; -2.25              ' 2.25-
PRINT USING "##"; 123                    ' %123
PRINT USING "[&] [!] [\  \]"; s; s; s     ' [ALMA] [A] [ALMA]
PRINT USING "## "; 1, 2, 3               '  1  2  3
```

Field	Meaning
`#`	a digit position; numbers are right-aligned and rounded to the decimals
`.`	decimal point; the `#`s after it are the decimals (at most 9)
`,`	before the point: thousands separators (each `,` is one position)
`+` at the start	always print the sign (`+` / `-`)
`-` at the end	print the minus sign after the number (a space when positive)
`&` / `!` / `\ \`	string: the whole string / its first character / exactly as many characters as the field is wide (`\` + spaces + `\`)
`_`	print the next character as it is (`_#`)

A number that does not fit is printed with `%` in front (`%123`); a lone leading 0 is dropped first (`#.##` with -0.5 → `-.50`). Other characters are printed as they are. The values cycle through the fields (the format starts again when it runs out); after the last value the text up to the next field is printed. A trailing `;` or `,` keeps the cursor on the line. The format must be a string literal; `**`, `$$` and `^^^^` fields are not supported. A program with `PRINT USING` includes the floating-point runtime (only the parts it needs).

chr$

```
print chr$(65)           # output character with PETSCII code 65 ('A')
var c = chr$(n)          # store byte value n in variable c
print ">" + chr$(42)     # usable in string concat
```

Branching

```
if x == 1 then
  print "YES"
else
  print "NO"
end
```

Select / Case

```
select x
  case 1:
    print "ONE"
  case 2:
    print "TWO"
  else:
    print "OTHER"
end
```

`select expr` evaluates the expression once and compares it against each `case` value in order. The first matching case body is executed and control jumps to after `end`. The optional `else:` body runs if no case matches. All values must be 8-bit (0–255).

Loops

```
loop 5               # counted loop (5 iterations)
  print "HI"
end

times 5              # alias for loop N ... end
  print "HI"
end
```

```
loop                # infinite loop
  x = x + 1
  if x == 5 then continue end  # skip to next iteration
  if x == 100 then break end
end

var i = 0
for i = 1 to 10      # for..next (preferred)
  if i == 5 then continue end  # skip to increment step
  print i
next

for i = 0 to 20 step 2
  print i
next i               # variable name after 'next' is optional

for i = 10 to 1 step -1     # counting down — negative constant step is supported
  print i
next

for i = 20 to 0 step -2     # terminates correctly at i = 0 (11 iterations)
  print i
next i

var i = 0
loop i = 1 to 10     # legacy loop..end syntax — identical code
  print i
end

while x < 100
  x = x + 1
end

repeat               # do-while: body runs at least once
  x = x + 1
until x == 100       # exits when condition is true
```

Direction rules for `for` / `loop`:

Case	Behavior
`for i = 1 to 10` (default `step +1`, `from ≤ to`)	counts up, standard
`for i = 0 to 20 step 2` (positive step, `from ≤ to`)	counts up by 2
`for i = 10 to 1 step -1` (negative constant step)	counts down; body runs for i = 10, 9, ..., 1
`for i = 20 to 0 step -2` (down to zero)	terminates at i = 0 by detecting the wrap-around below 0 after the decrement; no infinite loop

Case	Behavior
`for i = 10 to 1` (no step, `from > to`)	**compile-time error**: `for-loop: from (10) > to (1) with default step +1 loops 0 times — use 'step -1' to count down`

Code shape (1.6.0): the loop is *rotated* — `JMP test` (omitted when constant bounds guarantee a first pass), the body, `var += step`, then the test at the bottom branching back to the body, so there is no extra `JMP` per iteration. The step detects 8-bit wrap-around in both directions (`INC`/`BEQ` or `ADC`/`BCS` counting up, `BEQ`/`DEC` or `SBC`/`BCC` counting down), so `for i = 0 to 255` runs 256 times and stops, and `for i = N to 0 step -k` is always finite. Constant limits and steps are used as immediates (no zero page); a non-constant limit or step is evaluated once and kept in permanent zero page.

Non-constant `step` values (e.g. from a variable or expression) are treated as positive at compile time; if you need count-down with a runtime step value, split the loop or use a `while` construct.

Labels and goto

```
label main_loop
  x = x + 1
  if x < 10 then goto main_loop end
```

Forward `goto` (label defined later) is fully supported.

`gosub label` / `return` jump to a label and return back (JSR / RTS at machine code level). The label must be a `label name` statement, not a `sub` — it has no parameters and shares the same zero-page scope. `gosub` supports forward references (label defined after `gosub`).

```
gosub draw_border
...
label draw_border
  # ... draw something ...
  return              # RTS — returns to the instruction after gosub
```

Subroutines

```
sub greet()
  print "HELLO!"
end

sub set_color(col)
```

```
  color border col
  color text   col
end

greet()              # call with parens
call greet           # call keyword (no parens)
set_color(6)
```

Parameters are passed via dedicated zero-page slots. No recursion (slots are static).
Typed parameters are supported: `sub draw(x, y:int)` or `sub copy(src:string)` — string parameters receive a 2-byte pointer so the callee can index the source string via `src[i]`.

Functions (return values)

```
fn square(x)
  return x * x
end

fn add(a, b)
  return a + b
end

fn clamp(val, lo, hi)
  if val < lo then return lo end
  if val > hi then return hi end
  return val
end

var s = square(9)         # s = 81  — fn call as expression
var c = clamp(joy, 0, 39) # c = clamped value
print add(10, 20)         # 30  — usable inline in print
```

Functions support an optional `: word` return type for 16-bit values:

```
fn get_addr(): word
  return $C000
end

var ptr: word = get_addr()  # ptr = $C000
poke ptr, 42                 # STA (ptr),Y — valid indirect addressing
var v = peek(ptr)            # LDA (ptr),Y
```

`fn` is emitted in pass 2 (same as `sub`), so function bodies are never executed at startup. Forward references are fully supported.

QBasic-style `FUNCTION` / `SUB`

```
FUNCTION Negyzet (szam AS INTEGER) AS INTEGER
    Negyzet = szam * szam          ' return value = assign to the function name
END FUNCTION

FUNCTION Nagyobb (a AS BYTE, b AS BYTE) AS BYTE
    Nagyobb = a
    IF b > a THEN
        Nagyobb = b
        EXIT FUNCTION              ' return now, with the value assigned so far
    END IF
END FUNCTION

SUB Koszont (nev AS STRING)
    PRINT "Üdvözöllek, "; nev; "!"
END SUB

n = Negyzet(30)
Koszont "Zsolt"                    ' call without parentheses
Koszont("Zsolt")                   ' ... or with them
CALL Koszont("Zsolt")
```

- `function` is a synonym for `fn`; `p AS type` works in parameter lists and `AS type` after `)` gives the return type (same type names as `dim`). `: type` still works.
- Assigning to the function name stores into a hidden `<name>__result` variable that is returned at `END FUNCTION` / `EXIT FUNCTION`; `return expr` still works too.
- `END SUB`, `END FUNCTION`, `END IF`, `END SELECT` are accepted (the second word is optional); `EXIT SUB` / `EXIT FUNCTION` return (plain `exit` is still `bye`).
- In a SUB / FUNCTION closed by `END SUB` / `END FUNCTION`, an `END` alone on its line ends the program, as in QBasic — also inside an `IF … END IF` there. Close the blocks of such a routine QBasic-style (`END IF`, `END SELECT`, `WEND`, `LOOP`, `NEXT`); a routine closed by a plain `end` keeps the old block rules. `END` and `bye` inside any routine leave the program (before, they returned to the routine's caller).
- A sub/fn can be called without parentheses: `Koszont "Zsolt"`, `Rajzol x, y`.
- `PRINT a; b; c` — `;` joins items without spaces; a trailing `;` suppresses the newline. A `;` followed by a statement (`print x; y = 1`) still separates statements.
- Accented letters in string literals print as their base letter (`ü` → `u`, `ő` → `o`); the C64 charset has no accents.
- Word/float parameters now receive both bytes, and a string literal passed to a `string` parameter works (before, only the lo byte / a string variable worked).
- No recursion (parameters live in static zero-page slots); since 1.6.3 a recursive call is a compile error instead of silently wrong results.

More QBasic syntax (1.6.3)

The QBasic forms below compile next to the original syntax (`==` , `end` , `loop` , `#` comments all keep working).

```
' comment (QBasic style, like #)
IF x = 3 THEN PRINT "three"            ' single-line IF: no END IF needed
IF x <> 3 THEN PRINT "no" ELSE PRINT "yes"
IF n < 0 THEN
    PRINT "negative"
ELSEIF n = 0 THEN
    PRINT "zero"
ELSE
    PRINT "positive"
END IF

DO WHILE i < 10 : i = i + 1 : LOOP     ' also DO UNTIL c … LOOP,
DO : i = i - 1 : LOOP UNTIL i = 0      ' DO … LOOP WHILE c, DO … LOOP (endless)
WHILE k <> 5 : k = k + 1 : WEND
FOR i = 1 TO 100
    IF i = 12 THEN EXIT FOR             ' EXIT DO / EXIT FOR / EXIT WHILE
NEXT

SELECT CASE n
    CASE 1, 2, 3       : PRINT "small"
    CASE 10 TO 20      : PRINT "teen"
    CASE IS > 1000     : PRINT "big"
    CASE ELSE          : PRINT "other"
END SELECT

END                                    ' ends the program (screen is kept)
```

- **Conditions:** inside `IF` / `ELSEIF` / `WHILE` / `UNTIL` / `LOOP WHILE|UNTIL` , a single `=` compares. `<>` means not equal everywhere (same as `!=`).
- **Single-line IF:** a statement right after `THEN` makes a single-line IF that ends with the line, or with an `end` on the same line (`if c then x = 1 end` , as before). A multi-line IF needs a line break after `THEN` . As in QBasic, everything after `THEN` on that line belongs to the IF (`… THEN EXIT FOR : NEXT` would swallow the `NEXT`).
- **`END` on its own:** an `END` that does not close a block ends the program, like QBasic. A surplus `end` therefore no longer shows up as an error.
- **`DO` blocks:** inside a `DO` , a `LOOP` at the end of a line closes it (the original `loop N … end` / `loop i = … end` loops still work inside).
- **`SELECT CASE` :** the old `select x` / `case n:` form is unchanged. `SELECT CASE` is lowered to an IF chain over a hidden `select__N` variable.

Strings at runtime (1.6.3)

```
DIM s AS STRING, t AS STRING
s = "Pont: " + pont + "!"            ' numbers become text (16-bit, signed for INTEGER)
t = s                                ' a copy: changing s later does not change t
s = s + "?"
PRINT LEFT$(s, 4); RIGHT$(s, 2); MID$(s, 7, 3); MID$(s, 7)
PRINT LEN(s + t), VAL("1234") + 1, STR$(pont)
IF s = "ABC" OR t <> "" THEN PRINT "compare with = / <>"
```

- **Buffers:** a string variable that gets a computed value (`+` , `left$` , `right$` , `mid$` , numbers, another variable) owns an 80-character buffer; longer results are cut at 80. Assigning a literal still just points at it.
- **String arrays:** elements that get a non-literal value (or `input`) get their own 32-byte slot (31 characters), so `input nevek(i)` in a loop keeps every name.
- **Longer strings:** a line `option strings 255` lets every computed string variable hold up to 255 characters (C64 BASIC's limit); `option strings 200, 63` also gives string-array elements 63 characters (the slot is rounded up to 32, 64, 128 or 256 bytes). Every variable keeps its own buffer, so longer limits cost RAM: n + 1 bytes per string variable and slot size × elements per string array.
- **`STR$` / `VAL` :** `STR$` of a 16-bit or signed value has no leading zeros (`"-42"`). For compatibility, `STR$` of a byte keeps the 3-digit form (`"007"`) unless C64 BASIC spacing is on (`--basic-print` : `" 7"`). `VAL` reads 16-bit signed numbers (`VAL("1234") + 1` = 1235).
- **Comparison:** `=` / `<>` (and `==` / `!=`), and `<` `>` `<=` `>=` in PETSCII order like C64 BASIC; a string comes before every longer string it starts (`"AB" < "ABC"` , `"" < "A"`).

Signed INTEGER, INPUT, DOUBLE (1.6.3)

- **INTEGER is signed:** `DIM x AS INTEGER` is signed 16-bit (−32768 … 32767, like QBasic): printing, `<` `>` comparisons, `/` (truncates toward zero), `ABS` , text conversion, `INPUT` and `FOR` loops through zero (`FOR i = 3 TO -3 STEP -2`) all follow the sign. `WORD` stays unsigned 0 … 65535. Unary minus works anywhere (`a < -10` , `x * -2`).
- **`FOR` with a 16-bit counter** (`INTEGER` / `WORD`) is a real 16-bit loop. It used to wrap at 256, so `FOR k = 0 TO 300` ran 45 times.
- **`INPUT` :** into an INTEGER / WORD reads up to 6 characters (with `-` for INTEGER), into a DOUBLE accepts `12.5` (two decimals). `INPUT t(i)` reads into an array element, and `INPUT "prompt"; x` works too.
- **DOUBLE ÷ DOUBLE** is exact now (`7.5 / 2.5 = 3.00`); before, only the divisor's integer part was used. DOUBLE × DOUBLE was already supported.
- **Arrays over 256 bytes** (`DIM t(200) AS INTEGER` , `DIM g(19, 19)`) work with variable indices; the index used to wrap at 256 and write into the wrong element.
- **Recursion** (a sub/function calling itself, directly or through others) is a compile error. It used to compile and return wrong results, because parameters live in fixed zero-page slots.

Arrays

```
var scores = array(8)    # 8 bytes at $C000
                         # (all arrays are zero-filled once at program start — new in 1.5.7)

scores[0] = 100          # constant index → STA $C000
scores[i] = 99           # variable index → STA (ptr),Y
var v = scores[i]        # LDA (ptr),Y
print scores[2]          # usable inline in print

var moments = array_word(8)  # 16 bytes (8×2) at $C000+

moments[0] = $1234         # constant index → STA $C000 (lo), STA $C001 (hi)
moments[i] = $5678         # variable index → ASL A for stride; (ptr),Y × 2
var t: word = moments[1]   # LDA $C002, LDA $C003
```

Multi-dimensional arrays (new in 1.5.3)

Arrays may be declared with more than one dimension. They are stored **row-major** and indexed with a comma-separated subscript list. The total size is the product of all dimensions.

```
var grid = array(8, 8)     # 8×8 = 64 bytes at $C000
grid[1, 0] = 11            # row 1, col 0 → flat index 1*8+0 = 8 → STA $C008
grid[0, 1] = 22            # row 0, col 1 → flat index 1       → STA $C001
grid[r, c] = 33            # variable → computes r*8 + c, then STA (ptr),Y
var v = grid[r, c]         # LDA (ptr),Y

var wm = array_word(4, 4)  # 4×4 word array = 32 bytes
wm[1, 2] = $ABCD           # element 1*4+2 = 6 → bytes $C00C/$C00D

const ROWS = 3
const COLS = 5
var m = array(ROWS, COLS)  # dimensions may be compile-time constants
```

Row-major layout means the last subscript is contiguous: `grid[r, c]` sits at `base + r*COLS + c`. Any number of dimensions is supported (`array(a, b, c)`). When every subscript is a constant the address is folded at compile time to a single absolute store/load; a variable subscript emits the `row*stride + col` computation and an indexed `(ptr),Y` access. A single subscript into a multi-dimensional array is still allowed and treated as a flat/linear index (`grid[10]`). Dimensions must be compile-time constants (literals or `const`s), and the array must be declared before it is indexed.

Struct types (`type ... endtype`, new in 1.5.4)

Define a fixed layout of named fields with `type`, then allocate an array of instances the same way as a regular array. Field access uses `arr[idx].field` and works for both constant and variable indices.

```
type TEntity
  var x:  int      # 1 byte
  var y:  int      # 1 byte
  var hp: word     # 2 bytes  → element size = 4
endtype

const N = 8
var enemies: TEntity = array(N)   # allocates N × 4 = 32 bytes at $C000

enemies[0].x  = 5
enemies[0].y  = 10
enemies[0].hp = $0100
var v = enemies[0].hp             # read back — LDA $C002 / LDA $C003

var i: int = 2
enemies[i].x = 42                 # variable index → idx * elem_size + offset

# Float field (Q8.8 fixed-point)
type TBall
  var bx:  int     # 1 byte
  var by:  int     # 1 byte
  var vel: float   # 2 bytes Q8.8  → element size = 4
endtype

var balls: TBall = array(3)
balls[0].vel = 1.5                # stores Q8.8: hi=1, lo=128
var f: float = balls[0].vel       # read float field into float variable
print balls[0].vel                # prints "1.50"
```

Defaults, strings, nested types, parameters (after 1.6.6):

```
TYPE TPoint                       ' QBasic form: TYPE … END TYPE, "name AS type"
    x AS BYTE = 3                 ' default value
    y AS WORD = 1000
END TYPE

type TEntity
  var pos: TPoint                 # nested struct: fields pos.x, pos.y
  var hp: word = 100
  var name: string * 6 = "NOBODY"  # fixed-length string inside the element (7 bytes)
endtype

var e: TEntity = array(100)       # every element starts with the defaults
```

```
e[5].pos.x = 7
e[5].name = "ZS" + "OLT"          # computed text is copied in, cut to 6 characters
print e[5].name; " "; e[5].hp

sub heal(p AS TEntity, amount AS WORD)   # or  p: TEntity
  p.hp = p.hp + amount            # the element passed is changed
  p.pos.x = p.pos.x + 1
end sub
heal(e[5], 25)                    # pass an element
```

Rules and constraints:

- Field types: `int` / `byte` (1 byte), `word` / `integer` (2 bytes LE, unsigned), `float` (2 bytes Q8.8), `string * n` (a fixed-length string of up to n characters, 1–254, n + 1 bytes in the element; `string` alone is `string * 15`), and another struct type (nested: `arr[i].pos.x`). Fields are written `var name: type` or, QBasic style, `name AS type`; `END TYPE` closes the block like `endtype`.
- **Defaults:** `= constant` after a field (a number, a decimal for `float`, a string literal for `string`). They are the array's contents at program start (like `data name: …`); a nested field takes the defaults of its type. The element count must be a constant then.
- **String fields** work like string values: assign literals or computed strings (cut to the field length), print them, use them in `+`, `=` / `<>` / `<` and string functions.
- **Struct parameters:** `sub f(p AS TName)` / `p: TName` receives the address of the element passed (`f(enemies[i])`), so `p.field` reads and writes that element; a struct parameter can be passed on to another routine (`g(p)`). A function may take them too. `arr[i]` of a struct array without a field is that address.
- Element size = sum of field widths (declaration order, no padding).
- Storage lives at `$C000+` alongside regular arrays, and shows up in the memory map as an ordinary array of the total byte size.
- Constant index → `LDA/STA absolute` at compile time.
- Variable index → 16-bit `idx * elem_size` (before, it was computed in 8 bits and elements past offset 255 were wrong), then indexed `(ptr),Y` access.
- Only one subscript is allowed on a struct-typed array; multi-dimensional struct arrays are not supported (use a computed flat index).

See `examples/type_demo.ub`.

16-bit (word) variables

```
var ptr: word = $0400    # two ZP bytes: lo=$00 hi=$04
poke ptr, 6              # STA (ptr),Y
var v = peek(ptr)        # LDA (ptr),Y
```

16-bit products (fixed in 1.5.7). A product assigned to a `word` keeps all 16 bits, also when neither factor is a `word`:

```
var score: word = 0
var level = 3
score += 300 * level     # 900 (before 1.5.7 this was truncated to 8 bits: 44)
score = score + 100 * level
var big: word = 100 * level + 900
```

Since 1.6.1 this also holds inside sums and for `shl`, and for constant multipliers above 255: `py = cy * 8 + fy`, `py = (cy shl 3) + fy` and `w = cy * 300` keep all 16 bits (before, the sum forms were truncated to 8 bits). Byte targets keep 8-bit results.

Bitmap graphics

```
graphics on              # VIC-II hires bitmap mode (320×200, 1bpp); bitmap at $2000
graphics on multi        # VIC-II multicolor bitmap mode (160×200, 2bpp, 4 colours/cell)
graphics off             # return to text mode

gcls                     # clear bitmap (fills $2000-$3FFF) + set video matrix colors

plot x, y                # set pixel at (x, y);  x: 0-319,  y: 0-199
plot erase x, y          # clear pixel (AND ~mask)
plot xor x, y            # toggle pixel (EOR mask) — flicker-free animation
circle x, y, r           # midpoint circle centered at (x, y) with radius r; clips off-screen points
line x1, y1, x2, y2      # Bresenham line from (x1,y1) to (x2,y2); x: 0-319, y: 0-199
line erase x1, y1, x2, y2  # clear pixels along the line (AND ~mask)
line xor x1, y1, x2, y2    # toggle pixels along the line (EOR mask)
rect x1, y1, x2, y2      # draw rectangle outline (4 edges); x: 0-319, y: 0-199
rect erase x1, y1, x2, y2  # clear rectangle outline (AND ~mask)
rect xor x1, y1, x2, y2    # XOR rectangle outline (EOR mask)
paint x, y               # 4-connected flood fill from (x, y); fills clear pixels bounded by set ones

color pen c              # set the hires drawing color (0-15); see below

# ── Multicolor bitmap (graphics on multi, 160×200) ──
mplot x, y, color        # set multicolor pixel (x: 0-159, y: 0-199, color: 0-3)
mline x1, y1, x2, y2, color   # multicolor line
mrect x1, y1, x2, y2, color   # multicolor rectangle outline
mcircle x, y, r, color        # multicolor circle
```

Both `graphics on` variants blank the display (`LDA $D011 / AND #$EF / STA $D011`) while switching VIC registers, then re-enable it in the target mode — prevents mode-switch glitches.

`x` may be the full `0–319` range. Coordinates above 255 are handled automatically (the helper adds the 9th X bit), so `plot`, `line`, `circle` and `rect` all reach the right edge of the screen. Use a `word` variable when an X coordinate can exceed 255.

Hires drawing color — `color pen`

In hires mode (320×200) color is **per 8×8 cell**, held in the video matrix (high nibble = foreground, low nibble = background), not per pixel. `color pen c` sets a persistent foreground color (0-15) that `plot`, `line`, `rect`, `circle` and `paint` stamp into the cell of every pixel they draw; the cell's background nibble is preserved. It stays in effect until the next `color pen`. The default is white (1), so programs that never call `color pen` look exactly as before.

```
graphics on
gcls
color pen 2              # red
line 0, 0, 100, 100
color pen 6              # blue
circle 160, 100, 40
display on
```

Multicolor shapes — `mline` / `mrect` / `mcircle`

In multicolor mode (`graphics on multi`, 160×200) each pixel picks one of four color sources via a 2-bit value (`%00` background `$D021`, `%01` screen hi nibble, `%10` screen lo nibble, `%11` color RAM). `mplot` sets a single such pixel; `mline`, `mrect` and `mcircle` draw shapes the same way — the trailing `color` argument is the 2-bit source (0-3), and the actual colors come from the cell palette (screen / color RAM) exactly as for `mplot`.

```
graphics on multi
gcls
mcircle 80, 100, 40, 1
mrect 10, 10, 150, 190, 2
mline 0, 0, 159, 199, 3
display on
```

`mline` / `mrect` / `mcircle` reuse the same Bresenham / midpoint routines as their hires counterparts, plotting each pixel through `mplot`; off-screen points (x ≥ 160 or y ≥ 200) are skipped.

Double-buffered bitmap (flicker-free)

```
graphics on double       # double-buffered hires bitmap (320×200)
gcls                     # clears the HIDDEN back buffer
line 0, 0, 319, 199      # all drawing (plot/line/circle/rect/paint/gcls) goes to the back buffer
flip                     # show the drawn buffer; redirect drawing to the other one
graphics off             # back to text mode (restores VIC bank 0)
```

`graphics on double` keeps **two** complete hires bitmaps and shows only finished frames, eliminating flicker without any XOR tricks — each frame you `gcls`, draw, then `flip`.

Buffer	Bitmap	Video matrix	VIC bank	`$DD00` low bits
A (front, shown first)	`$2000`	`$0400`	bank 0	`%11`
B (back, drawn first)	`$6000`	`$4400`	bank 1	`%10`

- All pixel commands write through a runtime **draw base** (a zero-page byte), so they automatically target whichever buffer is currently hidden.
- `flip` waits for the lower border (raster ≥ 251) before switching the VIC bank, so the swap is **tear-free**, then points the draw base at the now-hidden buffer.
- Typical loop: `gcls` → draw the frame → `flip`. Call `display on` once after the first `flip` so the first shown buffer is already complete.

Requirements / limits:

- Hires only (not `multi`). Sprites are not fetched from the back buffer's VIC bank.
- The back buffer uses `$4000–$7FFF` (matrix `$4400`, bitmap `$6000–$7FFF`), so the program's machine code must stay below `$4400`. Most demos are only a few KB, so this is automatic; very large programs cannot use double buffering.
- On a stock 1 MHz C64 the per-frame full `gcls` (8 KB) limits the frame rate; on the Commodore 64 Ultimate raise `speed` for smooth high-rate animation.

See `examples/cube_demo.ub` — a tumbling 3D wireframe cube rendered flicker-free.

Block graphics (80×50)

```
graphics on block        # 80×50 block-pixel mode (text mode + custom 4-pixel charset @ $2800)
graphics off             # return to text mode
gcls                     # clear block playfield: screen RAM $0400-$07FF + color RAM $D800-$DBFF

plot4 x, y               # set block pixel at (x, y);  x: 0-79, y: 0-49
plot4 erase x, y         # clear block pixel at (x, y)
circle4 x, y, r          # draw midpoint circle in block pixels; clips to 80×50
```

A chunky low-res mode layered on standard 40×25 text. A 16-character custom charset copied to `$2800` encodes a 2×2 quadrant grid per character (bit3=TL, bit2=TR, bit1=BL, bit0=BR), so each text cell holds 2×2 block pixels → an effective 80×50 grid. No bitmap RAM is used (`$2000-$3FFF` stays free), making it faster than hires bitmap. `plot4` OR's the quadrant bit into the cell so overlapping pixels accumulate; `plot4 erase` clears it. `circle4` uses the same block-pixel helper to draw an outline circle in the 80×50 coordinate space. `gcls` clears both screen and color RAM. See `examples/block_demo.ub` .

Screen and color

```
cls                     # clear screen (KERNAL $E544)
cls fast                # fast fill: screen RAM + color RAM + HOME

color text 14           # text color register $0286
color border 6          # $D020
color bg 0              # $D021
color pen 2             # hires drawing color (0-15) for plot/line/rect/circle/paint

screen 0, 0, 65         # write char 65 ('A') to screen RAM at col 0, row 0 ($0400)
screen 10, 5, ch        # col 10, row 5 — col/row can be variables
screen 5, 3, 42, 7      # char 42 at col 5, row 3, color 7 (writes color RAM $D800 too)
screen x, y, ch, col    # all four arguments as variables

display on              # re-enable VIC display ($D011 DEN bit)
display off             # blank display

lowercase               # CHR$(14) → switch VIC-II to lowercase/uppercase charset
uppercase               # CHR$(142) → switch VIC-II back to uppercase/graphics charset

cursor 20, 10           # move cursor to col 20, row 10 (KERNAL PLOT $FFF0)
cursor x, y             # column from variable x (0–39), row from y (0–24)

print at 20, 10, "HELLO" # cursor(20,10) + print in one statement (no trailing newline)
print at x, y, "Score:", score  # any mix of exprs
print at 0, 0           # position only (no text, no newline)

scroll x 3              # set horizontal fine scroll: $D016 bits 0-2 = 3 (0-7)
scroll y 2              # set vertical fine scroll:   $D011 bits 0-2 = 2 (0-7)
scroll x n              # value can be a variable or expression (masked to bits 0-2)
scroll x 7 narrow       # set fine scroll and force 38-column mode (hide edge column)
scroll x 0 wide         # set fine scroll and restore 40-column mode
scroll row 12 left      # shift screen RAM row 12 left by one character
```

`lowercase` emits `LDA #$0E; JSR $FFD2` at runtime. String literals compiled after `lowercase` have their case swapped automatically — uppercase source chars are stored in the PETSCII lowercase slot (`$61+`) and lowercase source chars in the uppercase slot (`$41+`) — so `"Hello World"` source displays as **Hello World** on screen. `uppercase` emits `LDA #$8E; JSR $FFD2` and reverts to direct mapping. `cls` does **not** reset the charset mode.

`scroll x n` writes `(n AND 7)` into bits 0-2 of `$D016` (preserving bits 3-7).
`scroll x n narrow` writes the fine-scroll bits and clears `$D016` bit 3 (38-column mode).
`scroll x n wide` writes the fine-scroll bits and sets `$D016` bit 3 (40-column mode).
`scroll y n` writes `(n AND 7)` into bits 0-2 of `$D011` (preserving bits 3-7).
`scroll row R left` shifts one constant screen row left; write the new rightmost character with `screen 39, R, ch`.
Useful for smooth hardware scrolling: decrement each frame from 7 down to 0, shift screen RAM, reset to 7.

`screen col, row, char [, color]` writes directly to screen RAM (`$0400 + row*40 + col`) and optionally to color RAM (`$D800 + row*40 + col`). Constant col/row: address computed at compile time.

Ultimate 64 — CPU Speed

```
speed 4              # set CPU to 4 MHz  (reads $D031, updates bits 0-3, writes back)
speed 48             # 48 MHz  (maximum speed on U64)
speed max            # same as speed 48  (alias)
speed off            # back to 1 MHz  (alias for speed 1)

badlines on          # enable badline timing  ($D031 bit 7 = 0, default C64 behaviour)
badlines off         # disable badline timing ($D031 bit 7 = 1, more CPU cycles)

var t = turbo()      # 1 if turbo is active (bits 0-3 of $D031 != 0), 0 if at 1 MHz
```

Available MHz values: `1, 2, 3, 4, 5, 6, 8, 10, 12, 14, 16, 20, 24, 32, 40, 48`.
Constant values are rounded down to the nearest available speed at compile time.
Variable values are treated as a raw speed index (0–15) and OR'd into bits 0-3 of `$D031`.

$D031 index	MHz (U64)	MHz (U64 Elite-II)
0	1	1
3	4	4
6	8	10
11	20	24
15	48	64

Requires **U64 Turbo Control** set to `U64 Turbo Registers` or `Turbo Enable Bit` in the U64 config menu.
On a regular C64 or emulator without the register, the `poke` to `$D031` is silently ignored.

Keyboard

```
var key = getch()        # busy-wait on $FFE4 until key; returns PETSCII code
var k   = inkey()        # non-blocking: returns PETSCII code, or 0 if no key pressed
waitkey                  # wait until any key is pressed (CIA1 matrix scan; works during IRQ)
var k   = waitkey()      # same but returns raw $DC01 column bits (0 bit = key pressed)
var j = joy(2)           # read joystick port 2; returns inverted bits 0-4
```

```
var j = joy(1)          # read joystick port 1
                        # bit0=up(1) bit1=down(2) bit2=left(4) bit3=right(8) bit4=fire(16)
var mx = mouse_x()      # 1351 mouse: accumulated X position (helper: charge+delay+delta)
var my = mouse_y()      # 1351 mouse: accumulated Y position (EOR #$FF inverted)
var mb = mouse_btn()    # mouse buttons: reads $DC01 — bit0=left (fire), bit1=right (up pin)
```

`mouse_x()` / `mouse_y()` internally run a helper subroutine that handles: CIA1 `$DC00` capacitor charging, ~516-cycle delay, SID POT X/Y register read (`$D419` / `$D41A`), signed 7-bit delta computation, Y-axis `EOR #$FF` inversion, and accumulated position tracking (permanent ZP state, 8 bytes). The helper samples POT twice per frame. `mouse_y()` does **not** call the helper (only `mouse_x()` does) — it just reads the cached `accum_y` to avoid double-update.

The helper maintains a 9-bit X accumulator (`accum_x` + `accum_x_hi` with carry/borrow). Use `var sx: word = mouse_x()` — the compiler stores both lo and hi bytes, so the `sprite` command correctly handles `$D010` for X positions beyond 255.

The helper stores its state in ZP (`$02`–`$09`). The init flag must be zero before the first call — zero the area with `fill $02, 7, 0` early in your program. Also disable CIA1 interrupts (`poke $DC0D, $7F`) to prevent the KERNAL IRQ from touching `$DC00` during keyboard scanning — that disturbs the POT reading.

See `examples/mouse_demo.ub` for a working 1351 mouse demo with sprite tracking and button feedback.

Exit

```
bye                     # JSR $E544 (clear screen), clear STOP flag, RTS to BASIC
exit                    # alias for bye
```

Timing

```
wait 50                 # wait 50 raster-line transitions (~3.2 ms)
wait raster 100         # spin until $D012 == 100 (raster-split effects)
delay 1                 # wait 1 PAL frame (1/50 s ≈ 20 ms)
delay 20                # wait 20 frames ≈ 0.4 s; n can be a variable (0–255)
```

`delay N` counts N complete PAL frames using raster line 200 as the frame boundary.

SID Sound

```
sound 0, $1CAD, 25       # voice 0, freq $1CAD (≈ middle C PAL), 25 frames duration
sound 1, freq_word, 50   # voice 1, freq from word var, 50 frames (1 s at 50 Hz)
sound 2, 0, 0            # voice 2, silence

sid volume 15            # master volume full ($D418 = $0F); range 0-15
sid volume 0             # silence (master volume = 0)
sid stop                 # zero all 25 SID registers ($D400–$D418) — complete silence
```

`sound <channel>, <freq>, <duration>` — duration in PAL frames (1/50 s each). `sound` **blocks**: the program waits until the note has finished (use `sfx` below inside a game loop).
Fixed ADSR: attack/decay `$09`, sustain/release `$F0`, sawtooth waveform.
Master volume `$D418` always set to `$0F`.

Fixed in 1.5.7: the wait loop watched raster line 0, which `$D012` (low 8 bits only) shows twice per frame (lines 0 and 256), so every note lasted **half** as long as requested — and two branches were patched over their opcode, which crashed the program for any duration other than 0. `sound` now waits on raster line 200 exactly like `delay`.

`sid volume N` writes N to `$D418`. Bits 0-3 = volume (0-15), bits 4-7 = filter mode.
`sid stop` emits a 10-byte zero-fill loop — faster than 25 individual pokes.

Non-blocking sound effects (`sfx`) — new in 1.5.7

```
sound 0, $2000, 25           # BLOCKS: the program waits 25 frames here
sfx 0, $2000, 25             # returns at once; the note fades out by itself
sfx 1, $0500, 8, 128         # optional wave: 16 triangle, 32 saw (default), 64 pulse, 128 noise
sfx 0, freq_word, 4, 16      # freq may be a constant, a word var or an 8-bit expression
```

`sfx <channel>, <freq>, <frames> [, <wave>]` programs one SID voice and returns immediately, so it is safe inside a game loop. There is no gate-off timer: the envelope is set to attack 0 / sustain 0 / release 0 and the **decay** is chosen at compile time from `frames` (1 frame = 20 ms, nearest SID decay time: 6, 24, 48, 72, 114, 168, 204, 240, 300, 750, 1500, 2400 ms ...), so the note dies away over roughly that time. It re-triggers the voice each call (gate off, then on) and sets master volume `$0F`.
`channel` (0-2), `frames` (1-255) and `wave` must be compile-time constants.

SID Music

```
load sid "tune.sid"          # embed SID music at its native load address
load sid "tune.sid", $2000   # override: embed at $2000 regardless of SID header
```

`load sid` reads a PSID or RSID file at **compile time**, strips the header, and appends the raw music bytes to the output `.prg`. After `load sid`, two compile-time constants become available:

Constant	Description
`sid_init`	Init routine address — call once with A = song number (0-based)
`sid_play`	Play routine address — call every frame (50 Hz PAL) from an IRQ handler

Both constants work anywhere a constant address is accepted: `sys`, `irq`, `poke`, expressions.

Typical usage:

```
load sid "music.sid"

sub music_irq()
  poke $D019, $FF       # ACK VIC raster IRQ
  sys sid_play          # advance one frame of music
  irq_exit              # JMP $EA81: restore A/X/Y + RTI (proper IRQ exit)
end

sys sid_init, 0         # initialise SID chip: A=0 → first sub-tune
irq music_irq, $C0      # raster IRQ at line $C0 → 50 Hz on PAL

sid volume 15           # master volume on
```

Notes:

- SID data is placed **after** all generated code, padded with zeros up to the load address. The compiler warns if the SID load address would overlap generated code.
- PSID v1 and v2 are supported. If the SID header's load address is 0, the first two data bytes are used as the address (PRG-style, little-endian).
- Only one `load sid` per program is meaningful (the last one wins).

Tracker tunes (`tune`)

```
tune                    # optional: tune at $C800  (default $8000)
  speed 6               # frames per row (50 Hz PAL)
  inst 0, $41, $09, $F0, $0800, $0064, $27, $1F
  inst 1, $10, $08, $A8, $0400, $0000, $00, $0F
  imod 0, 6, 3, 9, 40, $0200, $0E00   # vibrato + pulse width modulation (1.6.2)
  ifilt 0, $0500, 20, 1               # cutoff sweep up to $500, back and forth (1.6.2)
  igate 1, 2, 1, $09                  # gate timer, hard restart, first frame $09 (1.6.2)
  itab 1, 2, 0, ".. +0", ".. +4", ".. +7"   # arpeggio table (1.6.2)
  order 0, 1, 0         # pattern order (patterns may repeat)
```

```
  pat 0, 0, "C-4 00", "...", "E-4 00 V24", "...", "G-4 00", "... .. C00"
  pat 0, 1, "C-2 01", "...", "...", "...", "G-2 01 F03"
  pat 1, 0, "G-4 00", "...", "E-4 00", "...", "C-4 00"
end
music play                # or: sys sid_init / irq handler calling sys sid_play
```

`tune ... end` describes a 3-voice tracker tune in plain text (the same cell notation the Visual Assembler SID editor shows) and builds the player **at compile time**. It defines `sid_init` / `sid_play` exactly like `load sid`, so `music play`, `music stop`, `music pause` and `music resume` (and a manual `irq` handler calling `sys sid_play`) work unchanged.

Line	Meaning
`speed N`	Frames per tracker row (1-255).
`inst id, ctrl, ad, sr, pw, cutoff, resfilt, modevol`	One instrument: control register (waveform bits plus ring mod `$04` / sync `$02`; the gate bit is added by the player), attack/decay, sustain/release, 12-bit pulse width, 11-bit filter cutoff, resonance/filter-routing byte (`$D417` style: any routing bit = route the voice that plays this instrument) and mode/volume byte (`$D418` style, bit 7 = 3 OFF).
`imod id, vibdelay, vibspeed, vibdepth, pwmspeed, pwmmin, pwmmax`	(1.6.2) Instrument vibrato: after `vibdelay` frames a triangle LFO that turns every `vibspeed` frames (1-15, 0 = off), depth 0-15 (scaled to the note's semitone). Pulse width modulation: `pwmspeed` added per frame (0 = off), bouncing between `pwmmin` and `pwmmax`.
`ifilt id, cutend, sweep, pingpong`	(1.6.2) Filter cutoff sweep: from the `inst` cutoff toward `cutend` by `sweep` (0-127) per frame; `pingpong` 1 = back and forth.
`igate id, gatetimer, hardrestart, firstwave`	(1.6.2) The gate is released `gatetimer` frames before the next note on the voice (keep it below `speed`); `hardrestart` 1 also sets ADSR to 0 then. `firstwave` (e.g. `$09` = test + gate) is written on the note's first frame.
`itab id, speed, loop, "step", ...`	(1.6.2) Wave / arpeggio table, up to 32 steps: `"WW +n"`, `".. -n"` or `"WW =n"` - waveform in hex (`..` keeps it) and a note offset in semitones or an absolute note (`=60`). One step every `speed` frames, then back to step `loop` (255 = stay on the last step).
`order p, p, ...`	Pattern order. Lines accumulate, up to 255 steps.
`pat p, v, "cell", ...`	Up to 32 rows for voice `v` (0-2) of pattern `p` (0-7). Missing rows are empty.

A **cell** is `NOTE INST FX`: `C-4`, `D#3`, `Eb2` or `...` for no note, a two-digit hex instrument (`..` to leave it), and an optional effect - `Vxy` vibrato (x = hold rows, y = depth), `Uxx` / `Dxx` slide up / down, `C` note cut, `Fxx` set speed. An effect-only cell can be written as `"... .. F06"` or just `"F06"`.

Notes on the player:

- All player state lives **inside the blob** (absolute addressing) - it uses no zero page, so it cannot collide with UltimateBasic variables or temporaries, and it keeps working when the program returns to BASIC.
- Every frame (1.6.2) the player runs each voice's wave/arpeggio table, vibrato, pulse width modulation and gate timer / hard restart, and the cutoff sweep of the voice that last started a filter-routed instrument (it owns the filter's cutoff, resonance and mode; `$D417` routes

every voice whose current instrument is filter-routed). It is the same algorithm as the Visual Assembler SID editor's exported player and preview, frame for frame.
- The blob is placed at a fixed address (default `$8000`, or `tune at $addr`) and the `.prg` is padded up to it, like `load sid`. Keep it clear of your code, arrays (`$C000+`) and any bitmap/charset memory. A tune needs about 2.5 KB plus 12 bytes per row-column of data.
- At most 8 patterns (32 rows each); the order list can repeat them freely.
- `music play` uses the CIA1 timer at 50 Hz; do not combine it with another `load sid` / `tune` (the last one wins).

Music playback

`music play/stop/pause/resume` is a high-level alternative to the manual `sys sid_init` / `cia_timer` setup. Requires a prior `load sid` statement or a `tune ... end` block (both define `sid_init` / `sid_play`).

```
load sid "tune.sid"         # embed SID file (defines sid_init / sid_play)

music play                  # initialise sub-tune 0 + start CIA1 50 Hz IRQ
music play 1                # start from sub-tune 1 (song number 0-based)
music stop                  # stop playback + zero all 25 SID registers ($D400-$D418)
music pause                 # disable CIA1 timer A IRQ (music freezes, SID unchanged)
music resume                # re-enable CIA1 timer A IRQ (continues from pause point)
```

Statement	Effect
`music play [n]`	call `sid_init(n)`, configure CIA1 timer A at 19 656 cycles (~50 Hz PAL), install IRQ wrapper
`music stop`	disable CIA1 IRQ + zero all 25 SID registers
`music pause`	disable CIA1 IRQ (SID output remains frozen)
`music resume`	re-enable CIA1 IRQ (continues from pause point)

The IRQ wrapper (emitted once at the end of the program) does: ACK CIA1 timer A → `JSR sid_play` → `JMP $EA81`.

Sprites

```
sprite 0, x, y, $2000    # sprite 0: set X, Y position and data pointer
sprite 0, x, y           # without data pointer (keeps existing)
sprite on  0             # enable sprite 0 ($D015 |= bit0)
sprite off 0             # disable sprite 0 ($D015 &= ~bit0)
sprite color 0, 7        # sprite 0 color = yellow ($D027)
sprite multicolor 0, on  # enable multicolor mode for sprite 0 ($D01C |= bit0)
sprite multicolor 0, off # disable multicolor mode ($D01C &= ~bit0)
sprite expand x 0, on    # double width ($D01D |= bit0)
sprite expand x 0, off   # normal width ($D01D &= ~bit0)
sprite expand y 0, on    # double height ($D017 |= bit0)
sprite expand y 0, off   # normal height ($D017 &= ~bit0)
```

```
sprite priority 0, on    # behind background ($D01B |= bit0)
sprite priority 0, off   # in front of background ($D01B &= ~bit0)
var h = sprite_hit()     # sprite-sprite collision ($D01E, cleared on read)
var b = sprite_bg_hit()  # sprite-background collision ($D01F, cleared on read)
var x = sprite_x(0)      # read sprite 0 X position (lo byte, $D000)
var y = sprite_y(0)      # read sprite 0 Y position ($D001)
sprite_frame 0, $2000          # select one static sprite image
sprite_frame 0, $2000, frame   # select an animation frame from a frame sequence
```

X supports full 9-bit range (0–319): use a `word` variable for runtime values > 255.
Sprite data pointer: `data_addr` must be 64-byte aligned; stored as `addr >> 6` at `$07F8+id`.

Sprite animation with `sprite_frame`

`sprite_frame` is the command used to change the displayed image of a sprite during animation. It changes only the sprite's data pointer; it does **not** move the sprite, enable it, or advance frames automatically.

Store the animation images consecutively, with each 63-byte sprite image occupying one 64-byte-aligned slot. Pass the zero-based animation frame as the third argument:

```
sprite_frame sprite_id, first_frame_address, frame_number
```

For example, with a base address of `$2000`, frame 0 uses `$2000`, frame 1 uses `$2040`, frame 2 uses `$2080`, and so on. The program controls animation timing and wrapping:

```
var frame = 0
loop
  sprite_frame 0, $2000, frame
  frame = frame + 1
  if frame == 4 then frame = 0 end
  delay 5
end
```

The two-argument form, `sprite_frame id, address`, simply selects one static sprite image and remains backward compatible. Sprite position is still controlled by `sprite id,x,y`.

Software bounding-box collision

```
var touching = box_hit(left1, top1, right1, bottom1,
                       left2, top2, right2, bottom2)
```

`box_hit()` performs an axis-aligned bounding-box (AABB) test and returns `1` when the two rectangles overlap or touch, otherwise `0`. Unlike `sprite_hit()` and `sprite_bg_hit()`, it does not read or clear VIC-II collision registers. The eight arguments are arbitrary 8-bit expressions, so boxes may be smaller than the visible sprite artwork or may describe non-sprite game objects. Coordinates are inclusive; keep `left <= right` and `top <= bottom`.

Sprite definition

```
sprdef 0
  $00,$3C,$00,  $00,$FF,$00,  $03,$FF,$C0,  $07,$FF,$E0,
  $0F,$FF,$F0,  $0F,$FF,$F0,  $1F,$FF,$F8,  $1F,$FF,$F8,
  $1F,$FF,$F8,  $0F,$FF,$F0,  $0F,$FF,$F0,  $07,$FF,$E0,
  $03,$FF,$C0,  $00,$FF,$00,  $00,$3C,$00,  $00,$00,$00,
  $00,$00,$00,  $00,$00,$00,  $00,$00,$00,  $00,$00,$00,
  $00,$00,$00
end
```

`sprdef id ... end` embeds 63 sprite bytes at the next 64-byte-aligned address in the code segment, emits a `JMP` over them, and automatically sets `$07F8+id = data_addr >> 6`.
To use the same shape for multiple sprites, read back the pointer:

```
var pg = peek($07F8)   # pointer set by sprdef 0
poke $07F9, pg         # copy to sprites 1–7
```

Character tile maps (`.ubmap`)

```
map load "levels/world.ubmap"
map load "level.bin", $4000 # Level Editor project, data at $4000 (1.6.1)
map draw map_x, map_y       # draw a 40x25 viewport to screen/color RAM
map view px, py             # smooth double-buffered view at pixel px, py (1.6.1)

var tile = map_tile(x, y)   # read character code from the map
map set x, y, 42            # change character code in writable map data
```

```
var shade = map_color(x, y) # read cell color (0 when map has no color data)
map color x, y, 7           # change cell color when color data is present
```

`map load` resolves the filename relative to the `.ub` source file, validates it at compile time, and embeds its character and optional color arrays in writable program RAM. A later `map load` replaces the active map for subsequent map commands.

`map load` also accepts a VisualAssembler `me-map` `.bin` export directly:

```
map load "map-multicolor.bin"
```

The first 1000 bytes become the 40×25 character map and the next 1000 bytes become cell colors. Multicolor mode and `$D021-$D023` are read from the VisualAssembler metadata trailer, so no `.ubmap` conversion is required.

Level Editor projects (new in 1.6.1). `map load` also reads a VisualAssembler Level Editor project (`Save level (.bin)`) and composes its screens into one map: a level of W×H screens becomes a (W·40)×(H·25) map with colors, empty slots are spaces. Maps are limited to 255×255 cells, so a level may be at most 6 screens wide and 10 tall.

```
map load "level.bin"            # 2x5 screens -> one 80x125 map
```

Placing the map data (new in 1.6.1). `map load "file", addr` stores the map at a fixed address instead of inline in the code, like `incbin "file", addr` (the address must be above the generated code or inside an `org` gap). Characters come first, colors follow:

```
map load "level.bin", $4000     # 80x125 map: characters $4000-$6713, colors $6714-$8E1F
```

Smooth scrolling — `map view` (new in 1.6.1)

```
map load "level.bin", $4000
var px: word = 0
var py: word = 0
loop
  map view px, py               # once per frame, instead of `wait raster`
```

```
    # read the joystick, change px / py, game logic ...
end
```

`map view x, y` shows the map at pixel position `x, y` (word expressions; `0..(width-40)*8` and `0..(height-25)*8`, out-of-range values are clamped). Call it once per frame: it waits for the frame boundary (raster line 250), shows the position given by the previous call, then takes the new one, so a position appears one frame later.

How it works: the fine part uses XSCROLL / YSCROLL in `$D016` / `$D011` with 38 columns and 24 rows (the scrolled-in edge stays hidden); the char steps swap two screen buffers, `$0400` and `$3C00`. The position is predicted two frames ahead and the hidden buffer is built for the next char cell in two halves before the view gets there, so the swap never waits. Color RAM cannot be double-buffered: its top half is copied in the last old frame behind the raster beam, the bottom half right after the swap ahead of it (both with interrupts masked for a few raster lines), so nothing tears. A direction change right at a char step may hold the view for one or two frames while the buffer is rebuilt.

Rules:

- The program code and inline data must stay out of `$3C00-$3FFF` (second buffer); load big maps with an address (`map load "file", $4000`), otherwise compiling stops with an error.
- The first `map view` draws the whole view into `$0400`. After that the screen belongs to `map view`: `print` / `screen` / `cls` write `$0400` only and are overwritten.
- Sprites work as usual: `map view` copies the sprite pointers `$07F8-$07FF` to `$3FF8-$3FFF` every frame, so `sprdef` / `sprite_frame` show in both buffers. Keep sprite data in VIC bank 0 (`$0000-$3FFF`, not `$1000-$1FFF`), e.g. by putting `sprdef` before a big inline `map load`.
- The charset bits of `$D018` are kept (`charset on` works); the multicolor bit of `$D016` is kept as `map load` set it.
- Leave about 6000 cycles (~95 raster lines) of game logic per frame; a longer frame makes the view hold for a frame but never tears.
- Example: `examples/level_scroll_demo.ub` (joystick-driven, with a player sprite).

`map draw map_x, map_y` copies a 40×25 viewport beginning at the specified map cell to screen RAM `$0400` and, when present, color RAM `$D800`. The map must contain the whole requested viewport: keep `map_x <= width-40` and `map_y <= height-25`. Coordinates and dimensions are currently 8-bit (0–255).

Normal maps clear the text multicolor bit in `$D016`. Multicolor maps set it and load their three global colors into `$D021`, `$D022`, and `$D023`. Character maps contain character codes, not charset pixels; combine them with `charset` / `chardef` or another charset-loading method. In multicolor text mode, cell colors 8–15 select multicolor characters according to the VIC-II rules.

UBMP version 1 binary format

All multibyte integers are little-endian:

Offset	Size	Meaning
0	4	ASCII magic `UBMP`
4	1	Version, currently `1`
5	1	Flags: bit 0 = color array present, bit 1 = multicolor text mode
6	2	Map width, 1–255 cells
8	2	Map height, 1–255 cells
10	1	Background 0 (`$D021`), low nibble used
11	1	Multicolor 1 (`$D022`), low nibble used
12	1	Multicolor 2 (`$D023`), low nibble used
13	width×height	Row-major character codes
following	width×height	Optional row-major color nibbles when flag bit 0 is set

An UBMP file must have the exact length implied by its header. Invalid magic, version, dimensions, or length produces a compile-time error.

Koala Painter image import

```
koala load "pictures/title.kla"  # validate and embed at compile time
koala show                       # enter bitmap multicolor mode
koala hide                       # return to the default text display
```

`koala load` accepts a standard 10003-byte Koala file (`$6000` load address plus 10001 data bytes) or a raw 10001-byte payload. Paths are relative to the `.ub` file. The payload contains 8000 bitmap bytes, 1000 screen bytes, 1000 color nibbles, and one background-color byte.

The compiler stores it at `$6000-$8710` . `koala show` copies the bitmap to `$2000` , the screen matrix to `$0400` , colors to `$D800` , and enables bitmap multicolor mode. `koala hide` clears bitmap/multicolor mode and restores the default text layout.

Generated code and helpers must finish below `$2000` , because the displayed bitmap overwrites `$2000-$3F3F` ; the compiler reports an error otherwise. Koala import cannot currently be combined with `load sid` in the same program.

Code layout: `org` — new in 1.5.7

```
sub a() ... end          # subs/fns are placed one after another from the end of the main body
org $4000                # the next subs/fns continue at $4000; $xxxx-$3FFF stays zero-filled
```

```
sub b() ... end
incbin "font.bin", $3800 # data can live in the gap ($3800-$3FFF is a whole VIC charset slot)
```

`org addr` goes **between sub/fn definitions** (it is ignored in the main body and only acts in the second pass). Subroutines never fall through into each other, so the skipped bytes are just zero-filled and reserved: an `incbin "f", addr` that lies wholly inside the gap is written into it, and `charset on` does not count the gap as code. The zero bytes are part of the `.prg`. `org` below the current code end is a compile-time error. Everything generated after the last sub (helpers, `data` tables) follows the last sub, i.e. lands above the gap.

Custom charset (`charset on` / `charset off` new in 1.5.7)

```
charset $3800            # set base address for chardef (default $3800)

chardef 65               # redefine character 65 ('A')
  $18,$3C,$66,$7E,$66,$66,$66,$00
end

chardef 66               # fewer than 8 bytes are zero-padded
  $7C,$66,$7C,$66,$7C
end
```

`charset base` sets the destination address used by all subsequent `chardef` statements. `chardef id ... end` embeds 8 bytes inline in the code segment (preceded by a `JMP` to skip over them), then copies them to `charset_base + id*8` at runtime.
Values must be compile-time constants written as `$xx` or decimal. `%` **binary literals are not supported** by the lexer (the manual said so before 1.5.7 — that was wrong).

`charset addr` only sets the `chardef` destination; it emits no code. To make the VIC-II use the set, switch it on (and off again when you are done):

```
charset $3800
chardef 1  $FF,$81,$81,$81,$81,$81,$81,$FF  end  # box border
charset on          # VIC-II reads characters from $3800 (screen matrix untouched)
...
charset off         # back to the ROM set ($1800 if `lowercase` is active, else $1000)
```

`charset on` read-modify-writes `$D018` bits 1-3 from the last `charset addr` (default `$3800`); the screen-matrix bits are preserved. The address must be a multiple of `$800` inside VIC bank 0 (`$0000-$3FFF`), otherwise it is a compile-time error. Both statements use the `charset addr` that was compiled last, so keep the directive in the main program body

(subroutine bodies are compiled after it). The equivalent raw write is `poke $D018, $1A` for `$2800` or `poke $D018, $1E` for `$3800`.

Overlap check. `charset on` claims the whole 2 KB set and every `chardef` its 8 bytes. If the generated code reaches into that area the build stops with `the program code ($080D-$xxxx) overlaps the charset area $2800-$2FFF; move the charset ...`. Large programs therefore need the set above the code (`charset $3800`) — or use `org` (above) to let the code continue above the set. Data placed in an `org` gap (`incbin ..., addr`) is not counted as code.

Memory

```
poke $D020, 2            # STA $D020
poke addr_var, 6         # STA (addr_var),Y  — if addr_var is word type
var v = peek($D012)      # LDA $D012
var v = peek(addr_var)   # LDA (addr_var),Y  — if addr_var is word type

var w: word = peek16($C000)   # read 16-bit little-endian: lo=$C000, hi=$C001
poke16 $0314, $EA81           # write 16-bit little-endian: lo→$0314, hi→$0315
poke16 ptr, w                 # word var as address; word var as value

var k = 2
var p = peek($07F8 + k)       # LDA $07F8,Y — constant + byte variable
poke $07F8 + k, p             # STA $07F8,Y
poke ptr + k, 1               # word + byte: full 16-bit address
```

`peek16(addr)` reads two consecutive bytes (lo, hi) as a `word`. `poke16` writes lo then hi.

An address is always a 16-bit value. Since 1.6.5 an address expression such as `$07F8 + k` keeps the constant's high byte and the carry (before, it was computed in 8 bits and `peek($07F8 + k)` read `$00F8 + k`). This holds for `peek`, `poke`, `peek16`, `poke16`, `fill`, `memcopy`, `drawmem`, `save` and `reu`; `fill` / `memcopy` lengths such as `40 * rows` are 16-bit too. `peek(k)` with a byte variable reads address `$00kk` (it returned the value of `k` itself before 1.6.5).

Disk I/O

```
load "PROGRAM"           # KERNAL LOAD: loads file from device 8 to its native address
load "DATA", $C000       # loads file to a specific address
chain "GAME"             # load another program over this one and RUN it (see below)
load "DATA", ptr         # addr from word variable

save "DATA", $C000, 4096 # KERNAL SAVE from $C000, 4096 bytes → device 8
save "PROG", start, len  # addr and len from word/int variables
```

`load` calls KERNAL `SETNAM` + `SETLFS` + `LOAD` (`$FFBD` / `$FFBA` / `$FFD5`).
Without address: secondary address 1 (the file's own 2-byte header is the load address).
With address: secondary address 0 (the file is loaded to the given address, its header is skipped).
(Fixed in 1.5.9 — before, the two were swapped: `load "F"` loaded to `$0000` .)
`save` calls `SETNAM` + `SETLFS` + `SAVE` (`$FFBD` / `$FFBA` / `$FFD8`). Requires both `addr` and `len` .

Chaining to another program — `chain` (new in 1.5.9)

```
chain "GAME"            # load GAME from the drive we were started from and RUN it
chain "GAME", 9         # from drive 9 (any expression)
print "LOAD ERROR"      # only reached if the file could not be loaded
```

`chain` loads another program to its own load address — normally `$0801` , right over the running program — and starts it exactly as if `LOAD"GAME",8,1` and `RUN` had been typed. This is what a boot / title program needs. A plain `load` cannot do it: the loaded file would overwrite the code that is executing the `load` .

How it works:

- A 46-byte, position-independent loader plus the file name is copied to the cassette buffer (`$033C`), which neither an Ultimate Basic program nor an Exomizer-packed file loads over.
- Before that the machine is put back into a clean state: VIC IRQs off, SID volume 0, `IOINIT` (`$FDA3` : CIA timers, keyboard IRQ, serial bus), `RESTOR` (`$FF8A` : default IRQ/BRK/NMI and I/O vectors), `$01 = $37` . Custom raster IRQs, `music play` and `nmi` handlers stop. Screen, colours and VIC bank registers other than `$DD00` are left as they are, so a picture can stay on screen while the next program loads (`IOINIT` selects VIC bank 0).
- The loader calls `SETNAM` / `SETLFS` (secondary 1) / `LOAD` . Without a device argument it uses the drive the program itself was loaded from (`$BA`), or drive 8.
- After a successful load it sets the end-of-program pointer (`$2D/$2E`), re-initialises BASIC (`$E453` , `$E3BF` — a self-extracting cruncher such as Exomizer `sfx` may have used BASIC's zero page) and RUNs the program (`$A659` , `$A7AE`). It works for Ultimate Basic programs, crunched (`exomizer sfx sys`) files and ordinary BASIC programs alike.
- If the file cannot be loaded (not found, no drive), `chain` returns and the program continues with the next statement.

The file name must be a string literal of 1–16 characters.

Serial channel file I/O

```
open 1, 8, 2, "MYFILE"  # open logical file 1, device 8, secondary 2, name "MYFILE"
open 2, 4, 7            # open printer (device 4), no filename
open ch, dev, sec       # channel, device, secondary from variables

print# 1, "HELLO"       # send "HELLO"+CR to logical file 1
print# ch, x, "text"    # any mix of vars, strings — same as print but to file
```

```
close 1                 # close logical file 1
close ch                # channel from variable
```

`open` calls `SETNAM` ($FFBD) + `SETLFS` ($FFBA) + `OPEN` ($FFC0). Without a filename, SETNAM is called with length 0.
`print#` routes output via `CHKOUT` ($FFC9), CHROUT per char (+ trailing CR), then `CLRCHN` ($FFCC).
`close` puts the channel number in A and calls `CLOSE` ($FFC3).

Input

```
input score             # read up to 3 digits from keyboard → 8-bit int var
input "Name: ", name    # optional prompt string, then read line → string var
input "Score: ", score  # prompt + int input
```

`input` uses KERNAL BASIN (`$FFCF`) for blocking, echoed line input with DEL support.

- **Int var**: accepts only `0` – `9`, max 3 chars; converts to 8-bit value on CR.
- **INTEGER / WORD var** (1.6.3): up to 6 characters (`-` for INTEGER), 16-bit value.
- **FLOAT var** (1.6.3): digits and `.`, e.g. `12.5` → 12.50 (two decimals).
- **SINGLE / DOUBLE var** (1.6.4): any number, `-12.5`, `.5`, `3.25E2`.
- **Array element** (1.6.3): `input t(i)` / `input nevek(i)`; `input "prompt"; x` also works.
- **String var**: accepts up to 30 chars; stores as null-terminated string; ZP pair updated.

Float / Fixed-Point

`float` variables use Q8.8 fixed-point format: the high byte is the integer part (0–255)
and the low byte is the fractional part (0/256 … 255/256).

```
var f: float = 3.5      # 3.5 → hi=3, lo=128 (= 0x0380)
var g: float = 0        # integer 0 is promoted to 0.0 automatically

f = 1.5                 # Q8.8 literal assignment
f = f + 1.5             # 16-bit Q8.8 arithmetic (result: 3.0)
f = f + g               # float + float

var n = int(f)          # extract integer part (hi byte) → 8-bit int
print f                 # prints as "N.DD" (e.g. 3.5 → "3.50", 1.25 → "1.25")
```

Operation	Example	Notes
Literal	`3.5`, `0.25`, `1.0`	parsed as Q8.8 at compile time

Operation	Example	Notes
Integer promotion	`f = 5`	stores 5.0 (hi=5, lo=0)
Add/sub	`f + 1.5`, `f - g`	16-bit Q8.8 arithmetic
Extract int	`int(f)`	returns hi byte as 8-bit int
Print	`print f`	format "N.DD", always 2 fractional digits

Caveat: Arithmetic overflow wraps at 255.255 (no saturation). Multiplication and division of two float variables are not yet supported — use `int()` + integer arithmetic for those cases.

Printing float expressions (fixed in 1.5.7). `print` of an arithmetic expression that involves a `float` (for example `print a / 10` with `a: float = 1`) now prints the Q8.8 result as `N.DD`. Before 1.5.7 it printed the raw 16-bit value as an integer (`25` instead of `0.09`). The fraction is truncated, not rounded: 1/10 = 25/256 prints as `0.09`.

Floating point: `SINGLE` / `DOUBLE` (1.6.4)

`dim x as single` and `dim x as double` are real floating-point numbers — the 5-byte format of C64 BASIC: about 9 significant digits, range $\pm 2.9 \cdot 10^{-39}$ … $\pm 1.7 \cdot 10^{38}$. (`single` and `double` are the same type.) The fast 2-byte Q8.8 type is still available as `float`.

```
DIM r AS DOUBLE, t AS DOUBLE
DIM k(9) AS SINGLE                    ' arrays: 5 bytes per element
r = 2.5
t = ATN(1) * 4 * r ^ 2                ' 19.6349541
PRINT t; " "; SQR(2); " "; 1E10; " "; 1 / 3
FOR r = 0 TO 1 STEP 0.25 : PRINT r : NEXT
FUNCTION Atfogo (a AS DOUBLE, b AS DOUBLE) AS DOUBLE
    Atfogo = SQR(a * a + b * b)
END FUNCTION
```

Operators	`+ - * /` and `^` (power; right associative, binds tighter than unary minus: `-2^2` = -4)
Functions	`SQR`, `SIN`, `COS`, `TAN`, `ATN` (radians), `EXP`, `LOG` (natural), `INT` (floor), `FIX` (truncate), `CINT` (round), `ABS`, `SGN`, `CSNG` / `CDBL` (no-op), `VAL`, `RND`
`MOD`	`a MOD b` = `a - b * FIX(a / b)` (sign of `a`, like VB / C `fmod`): `7.5 MOD 2` = 1.5; `x MOD 0` = x
`RND`	in a floating-point statement `RND` / `RND()` is a random number 0 ≤ x < 1 (`d = RND`, `x = RND * 6`); `rnd(n)` stays the integer 0 … n-1
Literals	`1.5`, `.5`, `1E6`, `2.5e-3`, `70000`
Comparisons	`= <> < > <= >=` in `IF` / `WHILE` / `UNTIL` / `SELECT CASE` and as values
Printing	C64 BASIC style: `.5`, `-3.25`, `3.33333333`, `1E+10`, `1.5E-07` (no leading space unless `--basic-print`); fixed decimals with `PRINT USING`

Strings	`s = "X=" + x`, `STR$(x)`, `VAL(s)` reads `-1.25E3`
`FOR`	floating-point counter, limit and step (`STEP 0.1`, negative, or a variable)
`INPUT` / `READ`	`INPUT x` reads any number; `READ x` reads the next `data` byte
Sub / function	`p AS DOUBLE` parameters and `AS DOUBLE` results

Mixing with integers. An expression is computed in floating point when it contains a SINGLE / DOUBLE value, `^`, one of the functions above, or a literal only floating point can hold (`1E6`, `300.5`, `70000`). Once a program uses floating point, every decimal literal (`0.1`) is a floating-point constant too. Storing a floating-point value into a `byte`, `integer` or `word` **rounds** it (half away from zero: 2.5 → 3, -2.5 → -3); into a `float` it becomes Q8.8. An integer value assigned to a DOUBLE is converted exactly. Integer expressions keep their integer rules (`var c = a + b` is still a byte).

Notes

- A whole-number literal from 32768 to 65535 keeps its value in a statement that computes in floating point (`d = 40000`, `d + 50000`). A statement counts as floating point when it names a SINGLE / DOUBLE variable, array, parameter or function, or uses `^`, a float-only literal or one of the functions above; elsewhere such a literal is a 16-bit pattern as before (`poke 53280, 1`, `w = 40000` into a WORD). A variable that only became floating point through `var x = 2.5` does not count — write `40000.0` there.
- `7 / 2` is 3.5 when assigned to a DOUBLE; in integer context it is 3 as before.
- `AND` / `OR` / `XOR` round their operands to integers.
- `SIN` / `COS` of a byte value keep the old 0–255 table meaning (full circle = 256); with a SINGLE / DOUBLE argument they work in radians.
- IRQ / NMI handlers may use floating point: a sub named in `irq`, `nmi` or `cia_timer` that (directly or through the subs it calls) uses SINGLE / DOUBLE saves the runtime state at its start and restores it before `irq_exit` / `nmi_exit` (about 4–5 thousand cycles per interrupt). Handlers without floating point are not slowed down. Not covered: a handler set by address or word variable, or one that leaves with a plain `RTS` / `RTI`.
- Cost: the floating-point runtime is assembled by the compiler and appended after the program; only the routines the program uses are included (about 2 KB for `+ - * /` and `PRINT` up to about 5 KB with all functions and `PRINT USING`). Variables take 5 bytes in the array area, plus one 2-byte zero-page pointer for the runtime. A multiplication takes roughly 1–2 ms on a 1 MHz C64.

Variables in RAM, array memory (1.6.4)

- **Zero page full:** when the program's variables do not fit into permanent zero page (`$02-$4F`), the compiler builds it again and moves the variables that are used least in hot code to normal RAM (after the arrays). A statement that uses such a variable copies it to a zero-page slot and back, so programs behave the same, only these variables are a bit slower. Loop counters, parameters and anything an IRQ handler may touch stay in zero page. The memory map shows them as `RAM:$xxxx`. Running out of zero page is no longer an error except in extreme cases (`out of zero page` remains for those).
- **Array memory:** arrays (and spilled / floating-point variables) use `$C000-$CFFF`. When they need more than 4096 bytes, the area starts at `$A000` instead, giving 12 KB

(`$A000-$CFFF`): the BASIC ROM is switched off at program start (`$01` bit 0) and switched back on at every exit to BASIC (`end`, `bye`, end of program, `chain`), so the program still returns to a working BASIC. Programs that call BASIC ROM routines themselves (`sys $Axxx`) must not use more than 4 KB of arrays.

Math functions

```
var a = abs(x - 20)      # two's-complement absolute value
var b = min(x, 39)       # 8-bit minimum
var c = max(x, 0)        # 8-bit maximum
var s = sgn(score)       # 0 = zero, 1 = positive (1–127), $FF = negative (128–255)
var r = rnd()            # LCG pseudo-random 0-255; seed from raster line
                         # (0 <= x < 1 in a SINGLE / DOUBLE statement: d = rnd)
var r = rnd(10)          # LCG pseudo-random 0-9 (rnd() mod n; result 0..n-1)
var s = sin(angle)       # sine: angle 0-255 (full circle), returns 0-255 (center=128)
var c = cos(angle)       # cosine = sin(angle+64)
var v = clamp(x, 0, 39)  # 8-bit unsigned clamp: result = max(lo, min(hi, val))

print hex(n)             # print as 2-digit uppercase hex
print bin(n)             # print as 8-bit binary string
```

String functions

```
var n = len(msg)         # length of null-terminated string var (0–255)
var c = asc(msg)         # PETSCII code of first character (0 if empty)
var c = asc("A")         # compile-time: constant PETSCII code
var n = val(s)           # runtime: parse decimal PETSCII string → 8-bit int (e.g. "042" → 42)
var c = msg[i]           # string character at index i: PETSCII code of msg[i]
msg[i] = c               # write PETSCII byte c to string at index i — STA (ptr),Y
msg[0] = 72              # constant index → LDY #0; STA (ptr),Y
```

Number formatting

```
print hex(n)             # print as 2-digit uppercase hex
print bin(n)             # print as 8-bit binary string
print dec(n, 4)          # right-justified decimal in a field of 4 chars (e.g. 42 → "  42")
print dec(n, width)      # width can also be a variable
```

`dec(n, width)` pads the number on the left with spaces to fill `width` characters.
If the number has more digits than `width`, it is printed without padding (no truncation).
In non-print contexts `dec(n, w)` evaluates to `n` unchanged (same as `hex` / `bin`).

REU (RAM Expansion Unit)

```
var ok = reu_present()   # 1 if REU detected, 0 if not (write/read test on $DF04)
var ok = reudet()        # alias for reu_present()

reu stash c64addr, bank, reu_addr, len  # copy C64 → REU
reu fetch c64addr, bank, reu_addr, len  # copy REU → C64
reu swap  c64addr, bank, reu_addr, len  # swap between C64 and REU
```

`reu_present()` performs a write/read-back test on REU register `$DF04`. Without an REU the write is lost (open bus), so the read-back differs — reliably detects presence without touching any side-effecting command register.

Parameter	Width	Notes
`c64addr`	16-bit	C64 RAM start — constant, `word` var, or 8-bit expr
`bank`	8-bit	REU bank number (0–7 for a 512 KB unit)
`reu_addr`	16-bit	Offset within the REU bank
`len`	16-bit	Bytes to transfer (`0` = 65 536 in REU hardware)

REU registers: `$DF01` command (`$B0` stash / `$B1` fetch / `$B2` swap),
`$DF02–$DF03` C64 addr, `$DF04–$DF05` REU offset, `$DF06` bank, `$DF07–$DF08` length.
Transfer is synchronous (CPU halted during DMA).
Requires a real REU or VICE: **Settings** → **Hardware** → **RAM Expansion Module**.

Memory utilities

```
fill screen 32           # fill screen RAM $0400–$07FF with value 32 (space char)
fill color 1             # fill color RAM $D800–$DBFF with value 1 (white)

fill $0400, 1000, 32     # fill 1000 bytes starting at $0400 with value 32
fill addr, 256, 0        # addr can be word var
fill ptr, len_word, val  # all operands can be expressions / word vars

memcopy $C000, $0400, 256   # copy 256 bytes from $C000 → $0400
memcopy src_ptr, dst_ptr, 40 # word vars for source and destination

drawmem $C000, $0400, 8, 10, 40 # blit 8×10 rect from $C000 → screen at $0400, stride 40
drawmem src_ptr, dst_ptr, w, h, 40 # word vars for src/dst
```

Both `fill` and `memcopy` support 16-bit lengths (0–65535). Use `word` variables for lengths > 255.

`drawmem src, dst, width, height, stride` copies a 2-D rectangular block. `src` is read linearly (packed rows); `dst` advances by `stride` bytes between rows — use `40` ($28) for the C64 screen or color RAM (40 columns). Width, height and stride are all 8-bit values. `src` and `dst` may be constants, `word` variables, or 8-bit expressions.

Raster IRQ

```
irq my_handler           # raster IRQ at line 0, handler = sub name or address
irq my_handler, 100      # raster IRQ at raster line 100
irq $C800, 200           # handler at fixed address
irq addr_word            # handler address from a word variable
```

Sets up a raster IRQ via the BASIC soft vector (`$0314` / `$0315`): disables CIA1 timer IRQ, ACKs pending VIC IRQ, writes raster line to `$D012` , enables VIC raster IRQ (`$D01A=$01`), writes handler address, and re-enables interrupts.

The handler **must** end with `sys $EA81` (KERNAL end-of-IRQ) — plain `RTS` or `RTI` will corrupt the stack. ACK the VIC IRQ first:

```
sub my_handler()
  poke $D019, $FF      # ACK VIC IRQ
  # ... work here ...
  sys $EA81            # JMP to KERNAL end-of-IRQ
end
```

Forward references are supported (`irq my_handler` before the sub is defined).

A handler sub may use SINGLE / DOUBLE floating point: it saves and restores the runtime state around its body (see *Floating point*).

NMI handler

```
nmi my_nmi               # set NMI vector $0318/$0319 to handler sub or address

sub my_nmi()
  # ... NMI work here ...
  nmi_exit               # JMP $FE47 — proper NMI exit (restores A/X/Y + RTI)
end
```

`nmi handler` writes the handler address to the NMI soft vector (`$0318` / `$0319`). The hardware NMI vector at `$FFFA` points to the KERNAL NMI routine which branches through `$0318` . The handler **must** end with `nmi_exit` (emits `JMP $FE47`) — using plain `RTI` will corrupt the stack. Forward references supported.

CIA1 timer IRQ

```
cia_timer 19656, my_handler   # CIA1 timer A: fires every 19656 cycles (~50 Hz PAL)
cia_timer period, handler      # period can be a variable or expression
```

Sets up CIA1 timer A as a periodic IRQ source via the BASIC soft vector (`$0314` / `$0315`):

1. SEI — disable interrupts
2. `$DC0D = $7F` — disable all CIA1 IRQs
3. Load 16-bit period lo→ `$DC04` , hi→ `$DC05`
4. Write handler address to `$0314` / `$0315`
5. `$DC0D = $81` — enable CIA1 timer A IRQ
6. `$DC0E = $01` — start timer A in continuous mode
7. CLI — re-enable interrupts

The handler must end with `irq_exit` (or `sys $EA81`) and should ACK the CIA1 IRQ:

```
sub my_handler()
  poke $DC0D, $01      # ACK CIA1 timer A IRQ (read also clears it)
  # ... work here ...
  irq_exit             # JMP $EA81: restore A/X/Y + RTI
end
```

PAL timing: clock = 985 248 Hz. Period for 50 Hz = 985 248 / 50 = 19 705 cycles ≈ `$4CC9` . Forward references supported.

Error handling

```
onerr goto err_handler   # set KERNAL I/O error vector ($0300/$0301) to a label

...

label err_handler
  print "I/O ERROR"
  bye
```

`onerr goto label` writes the label address (lo, hi) to KERNAL locations `$0300` and `$0301`.
When a KERNAL I/O error occurs (e.g. a failed `load` or `open`) the KERNAL executes `JMP ($0300)`,
which branches to the label. Forward references (label defined after `onerr goto`) are supported.

Compile-time file embedding (`incbin` skip / length new in 1.5.7)

```
incbin "sprites.bin"            # embed raw binary bytes at current code position
incbin "charset.bin", $2000     # embed at an absolute address, padding as needed
incbin "font.prg", $3C00, 2       # ...skipping the first 2 bytes (e.g. a .prg load address)
incbin "big.bin", $4000, 16, 256  # ...skip 16 bytes, then take at most 256 bytes
include "defs.ub"        # inline another .ub source file (lexed+parsed in place)
```

`incbin "file", addr, skip [, length]` drops the first `skip` bytes of the file (typically the 2 byte load address of a `.prg`) and optionally takes at most `length` bytes; `skip` and `length` are constants, and a `skip` beyond the end of the file is an error. The address form requires the target to be above the generated code — or inside an `org` gap.

Data / Read

```
data 1, 2, 3, 255        # constant byte table
read varname             # load next byte into varname (auto-declares if needed)
```

All `data` values are collected at compile time. A 2-byte ZP pointer is automatically allocated and initialised at program start. Each `read` advances the pointer.

Initialised arrays — `data <array>: …` (new in 1.5.8)

```
var sinus = array(8)
data sinus: 128, 218, 255, 218
data sinus: 128, 37, 0, 37      # further lines continue where the previous one stopped
print sinus[i]                  # random access, like any array

var xpos = array_word(2)
data xpos: 24, 300              # word arrays take 16-bit values (stored lo, hi)
```

A `data` line whose first item is `name:` fills a declared `array` / `array_word` instead of the `read` stream. The values are stored after the code and copied into the array once at

program start (right after the arrays are zeroed), so the table is fully writable and indexed like any other array. More values than the array holds, or an undeclared name, is a compile error. These lines do not affect `read`.

Inline assembly

```
sys $FFD2               # JSR $FFD2
sys $FFD2, 7            # LDA #7 ; JSR $FFD2  (pass byte value in A register)
irq_exit                # JMP $EA81 — proper IRQ handler exit (restore A/X/Y + RTI)
asm $EA, $EA            # inline raw bytes (NOP NOP) — legacy form
asm {
  ; Full 6502 mnemonics and addressing modes
  LDA #$07              ; immediate
  STA $0286             ; absolute
  LDA $50               ; zero-page  ($50 ≤ $FF → ZP auto-selected)
  STA $0400,X           ; absolute,X indexed
  LDA ($50),Y           ; (indirect),Y
  LDA ($50,X)           ; (indirect,X)
  JSR $FFD2             ; subroutine call
  JMP $C000             ; absolute jump
  JMP ($FFFC)           ; indirect jump

  CLC
  ADC #1
  SEC
  SBC #1

  TAX                   ; implied / transfer
  ASL A                 ; accumulator (also just: ASL)
  LSR A
  ROL
  ROR

  ; Branches — operand is an absolute address; offset is computed automatically
  BNE loop              ; forward or backward branch to local label
  BEQ done

loop:
  NOP
done:
  RTS

  ; #<label / #>label — lo / hi byte of a label address
  LDA #<handler
  STA $0314
  LDA #>handler
  STA $0315

  ; * — current assembly location
  JMP *
```

```
    ; Raw hex bytes (backward-compatible with old asm { $xx ... } syntax)
    $EA $EA                ; two NOP bytes
  }
```

Addressing modes:

Syntax	Mode	Bytes	Example
(no operand)	Implied	1	`NOP`, `RTS`
`A`	Accumulator	1	`ASL A`, `LSR`
`#value`	Immediate	2	`LDA #$07`
`$zz` (0–255)	Zero-page	2	`LDA $50`
`$zz,X`	ZP,X	2	`LDA $50,X`
`$zz,Y`	ZP,Y	2	`LDX $50,Y`
`$xxxx`	Absolute	3	`LDA $0400`
`$xxxx,X`	Absolute,X	3	`LDA $0400,X`
`$xxxx,Y`	Absolute,Y	3	`LDA $0400,Y`
`($xxxx)`	Indirect	3	`JMP ($FFFC)`
`($zz,X)`	(Indirect,X)	2	`LDA ($50,X)`
`($zz),Y`	(Indirect),Y	2	`LDA ($50),Y`
`label`	Relative	2	`BNE label` (branches only)

- `$zz` (1–2 hex digits, value ≤ 255) selects zero-page if the instruction supports it; otherwise auto-upgrades to absolute. Use `$00xx` (4 digits) to force absolute.
- Branch operands are absolute addresses; the relative byte offset is computed automatically.
- Local labels (`name:`) are scoped to the `asm { }` block. Forward branches resolved in pass 2.
- `#<label` / `#>label` yield the lo / hi byte of a label's address.
- `*` yields the current instruction address, so `JMP *` assembles as a self-loop.
- Lines starting with `$`, `%`, or a digit are emitted as raw bytes (backward-compatible).
- Inside `asm { }`, comments are `;` or `//` to end of line. (`#` is the immediate prefix, not a comment.)

Mixing `asm { }` with subroutine parameters

Parameter names are **not accessible** inside `asm { }` blocks. Use UltimateBasic statements to move values into known locations before the `asm { }` block:

```
sub set_colors(border_col, bg_col)
  poke $D020, border_col   # UltimateBasic resolves the ZP address
  poke $D021, bg_col
  asm {
    ; values are already in $D020 / $D021
    LDA $D020
```

```
    }
end
```

For routines whose entire body is assembly — especially IRQ handlers that must cross-reference each other — put all handlers in a **single top-level `asm { }` block** in the main program. Labels in the same block share scope, so `irq1` and `irq2` can reference each other freely. See `examples/raster_irq_demo.ub`.

String ↔ integer

```
numstr score, $0340      # writes "042\0" at $0340 (always 3 digits, zero-padded)
var n = str_to_int("42") # compile-time: Expr::Number(42)

print str$(score)               # print 8-bit int as 3-digit decimal string ("000"–"255")
print "Score: " + str$(score)   # usable in string concat print context
var s: string = str$(n)         # assign str$() result to a string var (shared static buffer)
```

`str$(n)` converts an 8-bit value to a 3-character decimal string (always 3 digits with leading zeros, e.g. `5` → `"005"`, `42` → `"042"`, `255` → `"255"`) followed by a null terminator.
The result pointer is stored in a permanent ZP pair allocated at compile time.

> **Note:** `str$(n)` uses a single shared 4-byte static buffer. Calling `str$(n)` again overwrites the previous result. For concurrent display of multiple values, use `numstr` to write to separate absolute addresses.

Examples

File	Description
`examples/features.ub`	const, label/goto, poke/peek, rnd, math functions
`examples/new_features.ub`	sub params, arrays, word vars, string vars
`examples/bitmap_demo.ub`	320×200 bitmap, plot, graphics on/off
`examples/block_demo.ub`	80×50 block graphics, plot4, circle4, graphics on block
`examples/joystick_demo.ub`	joystick reading, sprite movement
`examples/mux_demo.ub`	raster sprite multiplexer (3 windows × 8 sprites = 24)
`examples/orbit_demo.ub`	24-sprite orbit with pulsating radius and random colors
`examples/plasma_demo.ub`	plasma-effect bitmap with raster bar border animation
`examples/sprite_data.ub`	sprdef shape data (included by other demos)

File	Description
`examples/sprite_mux_orbit.ub`	24-sprite orbit demo with sprdef + precomputed positions
`examples/sprite_orbit_demo.ub`	8 hardware sprites in circular orbit via sin/cos table
`examples/reu_bitmap_demo.ub`	REU stash/fetch with bitmap graphics
`examples/sid_music_demo.ub`	SID music player with raster IRQ and keyboard exit
`examples/tenprint.ub`	5 TENPRINT maze implementations with menu; demos `lowercase` charset mode
`examples/countdown_demo.ub`	Count-down `for..next` with negative step, incl. `for i = 20 to 0 step -2`
`examples/countdown_errors_demo.ub`	Compile-time error for default-step `from > to`
`examples/explicit_demo.ub`	`--explicit` CLI-flag demo with fully typed `var`, `sub`, `fn`
`examples/explicit_errors_demo.ub`	Loose code that builds without `--explicit`, errors with it
`examples/dim_demo.ub`	`DIM … AS …` scalars and arrays of every type
`examples/qbasic_demo.ub`	QBasic-style FUNCTION / SUB / ELSEIF / DO-LOOP / SELECT CASE / strings
`examples/float_demo.ub`	SINGLE / DOUBLE floating point: `^`, SQR, SIN, ATN, EXP, LOG, `FOR … STEP 0.5`, `AS DOUBLE` function

CLI reference

```
ub build <input.ub> [OPTIONS]

  -o, --output <file>   Output .prg or .crt file (default: <input>.prg)
  -v, --verbose         Show zero-page layout and code hex dump
  --no-stub             Skip the BASIC SYS stub (code loads at $0801)
  --debug               Also produce .sym, .dbg and .vs debugger files
  --asm                 Also produce a readable 6502 codegen .asm listing
  --d64 [file]          Also produce a .d64 disk image;
                          without a filename defaults to <output>.d64
  --add <file>          Add an extra file to the .d64 disk image;
                          may be repeated for multiple files
  --explicit            Require :type on every var / sub-param / fn-param
  --basic-print         C64 BASIC PRINT spacing: spaces around numbers, ',' = 10-column zones
  -h, --help            Show help
```

Warnings

The compiler prints warnings (they do not stop the build) for a byte variable that gets a value it cannot hold:

```
line 5: warning: 'b' is a byte (0-255) and 1000 does not fit — it wraps; declare it 'as integer' / 'as word' if that is not intended
```

A result masked with `and`, `mod`, `/` or `shr` is not reported, and neither is byte + byte.

CRT export

Ultimate Basic can also write a Magic Desk type-19 cartridge image when the output filename ends in `.crt`:

```
ub build demo.ub -o demo.crt
```

The compiler wraps the generated PRG in a banked CRT container using the same layout as VisualAssembler's CRT export: a 64-byte cartridge header, an 8×8K Magic Desk image, and a boot loader in bank 0 that copies the PRG payload into C64 RAM before jumping to the program entry point. The `.prg` and `.d64` paths continue to work as before.

Explicit-type mode

Pass `--explicit` to force every declaration site to carry a `:type` annotation:

```
ub build game.ub --explicit
```

Without the flag, Ultimate Basic accepts both `var x = 5` (type inferred from the initializer or defaulted to `int`) and `var x: int = 5`. With the flag, only the annotated form compiles — the loose form becomes a compile-time error.

What it enforces:

Declaration form	Without `--explicit`	With `--explicit`
`var name = expr` (no `:type`)	ok — type inferred	error
`var name: int = expr` (or word/float/string)	ok	ok
`dim name` (no `as`)	ok — type inferred	error
`dim name as integer` (any type)	ok	ok
`var arr = array(N)`	ok — implicitly `array`	ok — unchanged

Declaration form	Without `--explicit`	With `--explicit`
`var arr = array_word(N)`	ok — implicitly `array_word`	ok — unchanged
`const NAME = value`	ok	ok — unchanged
`sub foo(a, b)` (untyped params)	ok	error per untyped param
`sub foo(a: int, b: int)`	ok	ok
`fn foo(a): int` (untyped param)	ok	error on the param
`fn foo(a: int): int`	ok	ok

Constants and array declarations are always accepted — their type is fixed by the declaration form, so `:type` would be redundant.

Example error output:

```
$ ub build myprog.ub --explicit
Compilation errors:
  line 14: 'explicit' mode: 'var loose' has no type — use 'var loose: int|word|float|string'
  line 16: 'explicit' mode: parameter 'a' has no type — use 'a: int|word|float|string'
```

The flag is a build-time switch — nothing in the source needs to change to opt in or out. Add it to your Makefile / build script to enforce typed style project-wide, or drop it for exploratory scripts. See `examples/explicit_demo.ub` and `examples/explicit_errors_demo.ub`.

Debug files

Use `--debug` to generate debugger symbols together with the program:

```
ub build demo.ub --debug
```

The compiler writes the files next to the `.prg`, using the output file's stem:

File	Format and purpose
`demo.sym`	KickAssembler-compatible symbol source, suitable for importing into assembler source
`demo.dbg`	C64Debugger/RetroDebugger KickAssembler debug-dump containing the program segment and labels
`demo.vs`	VICE monitor command file containing `al` commands for address labels

All three exports include `program_start`, `program_end`, variables, arrays, subroutines, and BASIC labels known after code generation. For example, load the

VICE symbols with its `-moncommands demo.vs` command-line option or the monitor's `ll "demo.vs"` command.

The current `.dbg` export provides segment and address-symbol information. It does not yet include instruction-to-source-line mappings for source-level stepping.

Assembly codegen listing (new in 1.5.2)

Use `--asm` to write a readable 6502 source listing next to the PRG:

```
ub build demo.ub --asm
```

For an output named `demo.prg`, this creates `demo.asm`. The listing is produced from metadata collected while Ultimate Basic generates the machine code; it is not merely a disassembly of the completed PRG. It contains:

- `; UB:` comments marking the generated byte range of each emitting UB statement;
- named constants for zero-page variables and `$C000+` arrays, including their types/sizes;
- final names and addresses for subroutines and BASIC labels;
- generated `loc_xxxx` labels for relative branches and in-program `JMP` / `JSR` targets;
- normal 6502 mnemonics and operands, with compiler symbols substituted where known;
- the exact C64 address and emitted bytes beside every instruction;
- compiler-generated helpers and embedded code in their final memory order;
- `.byte` output for bytes that do not decode as supported 6502 instructions.

Compiler helpers receive descriptive names such as `ub_helper_plot`, `ub_helper_line_erase`, `ub_helper_print_hex`, and `ub_helper_music_irq`. Known embedded data—including `data` statements, map character/color arrays, sprite and character definitions, the sine table, load filenames, `incbin` files, SID music, and Koala payloads—is emitted as named `.byte` regions. Long zero-filled address gaps use KickAssembler's `.fill` directive.

Example excerpt:

```
.label x                = $02 ; int

* = $080D

ub_start:
    cld                         ; $080D: D8
    ; UB: var x
    lda  #$01                   ; $080E: A9 01
    sta  x                      ; $0810: 85 02
```

The address/byte comments make the listing easy to compare with the `.prg`. The output uses KickAssembler syntax (`.label`, `.byte`, `.fill`, and `* = origin`) so it can be assembled again; machine-code comments do not affect the result.

`--add` requires `--d64`. The compiled `.ub` program is always the first file on the disk; each `--add` file is appended after it. File names on the disk are derived from the source file stem, uppercased (e.g. `music.prg` → `MUSIC`).

After a successful build the compiler always prints a memory map:

```
demo.ub → demo.prg  (386 bytes)

  Load:    $080D – $0989

  Variables (zero page):
    score    ZP:$08   byte
    lives    ZP:$0A   byte
    msg      ZP:$0C   string
    ptr      ZP:$0E   word

  Subroutines:
    greet    $0900
    show     $0912

  Arrays ($C000+):
    sprites  $C000   8 bytes
```

With `-v` the output additionally shows the internal ZP allocations and a full hex dump.

Generated code (optimised in 1.6.0)

The code generator emits tighter 6502 for the common cases; programs need no changes:

- Conditions in `if` / `while` / `until` / `select` are a `CMP` plus a branch (no 0/1 value is built and re-tested). `x > n` is compiled as `x >= n+1`; `and` / `or` of comparisons short-circuit when the right side has no side effects; `while` tests at the bottom of the loop.
- Constants and byte variables are direct operands (`ADC #n`, `CMP zp`); `poke` to a constant or `word` address stores directly.
- Arrays with a variable index use `base,Y` addressing instead of a zero-page pointer.
- When a statement starts by loading the variable the previous statement just stored, the redundant `LDA` is skipped.

Use `--asm` to see the generated code for your own program.

Known limitations

Feature	Limitation
Integer arithmetic	`var` / `byte` values are 8-bit unsigned (0–255) and wrap (`200 + 100` → 44 in a byte); a constant that cannot fit (`b = 300` , `b = a + 1000`) gives a compile-time warning; `integer` is signed 16-bit, `word` unsigned 16-bit
Zero page budget	Permanent zero page is `$02–$4F` (78 bytes): every variable and sub/fn parameter takes 2 bytes, every running `for` loop 2 more. Since 1.6.4 variables that do not fit move to RAM automatically (see *Variables in RAM*); only extreme cases still report `out of zero page`
Binary literals	`%01010101` (since 1.6.4), `$hex` and decimal
Subroutines	No recursion — ZP parameter slots are statically allocated (a recursive call is a compile error)
`END` in QBasic routines	In a SUB / FUNCTION closed by `END SUB` / `END FUNCTION` , an `END` alone on its line ends the program; blocks in such a routine must be closed QBasic-style (`END IF` , `WEND` , …), a UB-style `if … ⏎ end` there reads as program end
Struct types	One subscript per struct array (no `array(8, 8)` of a struct); an element is at most 256 bytes; `word` / `integer` fields are unsigned 16-bit; string fields are fixed-length (up to 254 characters); a struct parameter passes the element by reference
Floating point	`single` / `double` are 5-byte floating point since 1.6.4 (the routines a program uses, ~2–5 KB, are added to it); `float` stays Q8.8 (0–255.99)
Unary minus	Since 1.6.4 a leading `-` binds to its operand: `-5 - 3` is -8 (it was -(5 - 3) = -2)
String vars	Computed values live in an 80-character buffer per variable (longer results are cut), string-array element values in 31-character slots — `option strings n, m` raises both (up to 255); no shared heap; `<` / `>` order is PETSCII byte order
`rnd()` / `rnd(n)`	Simple LCG, not cryptographic; period = 256 (the floating-point `rnd` is a 32-bit xorshift)
`abs()` / `sgn()` / `min()` / `max()`	8-bit values only; `abs` / `sgn` treat values as signed (bit 7 = negative → `abs` two's-complements, `sgn` returns `$FF`); `min` / `max` are unsigned (0–255)
`plot`	Out-of-range pixels are silently clipped (Y ≥ 200 or X ≥ 320 → no-op)
`mplot`	No bounds checking — x must be 0–159, y must be 0–199
`mline` / `mrect`	Multicolor; x: 0–159, y: 0–199. Off-screen pixels wrap (no clipping) — keep coordinates in range
`mcircle`	Multicolor; clips off-screen points (x ≥ 160 or y ≥ 200 are skipped)
`color pen`	Hires only; sets the foreground nibble of touched cells (background preserved). Has no effect in block mode (`plot4` / `circle4`)
`rect`	No bounds checking — x: 0–319, y: 0–199; x1≤x2 and y1≤y2 not enforced (degenerate/inverted rects produce undefined output)
`plot4`	No bounds checking — x must be 0–79, y must be 0–49 (block mode)
`circle4`	Clips off-screen block pixels; useful radius is roughly 0–49 in 80×50 block mode
`chr$`	No PETSCII↔ASCII mapping — n is passed as-is to CHROUT
`music play`	Requires `load sid` or a `tune` block; only one CIA1 wrapper is emitted (last `music play` wins)
`graphics on double`	Hires only; uses `$4000–$7FFF` for the back buffer, so program code must stay below `$4400` ; not combinable with sprites or multicolor

Feature	Limitation
Error reporting	Compile-time only; `onerr goto` handles KERNAL I/O errors at runtime